

LOWL System Language Static Compiler Part 2



By Anthony Matarazzo (c) 2026

Source Code <https://github.com/amatarazzoconsulting/lowl>

Chapter 15: System Programming Builtins

15.1 Introduction to System Programming Builtins

lowl provides a comprehensive set of builtin functions that expose x86_64 hardware features directly to the programmer without requiring inline assembly. These builtins cover every aspect of system programming: interrupt control, port I/O, model-specific registers, control registers, performance monitoring, cache management, and atomic operations. Each builtin is implemented as a compiler intrinsic, meaning it generates the optimal instruction sequence for the target CPU. Unlike library functions that might incur call overhead or require linking, builtins are expanded inline at the call site. This chapter covers the complete set of system programming builtins, organized by functional area, with detailed explanations of the underlying hardware and practical examples.

15.2 Interrupt and Exception Control

Interrupt control builtins manage the processor's interrupt flag and execution suspension. The `disable_interrupts()` builtin executes the `cli` (Clear Interrupt Flag) instruction, which masks all maskable interrupts. The `enable_interrupts()` builtin executes the `sti` (Set Interrupt Flag) instruction, which re-enables interrupts. These builtins should be used carefully: long periods with interrupts disabled can cause system latency and missed hardware events. The `halt()` builtin executes the `hlt` instruction, which stops instruction execution until the next interrupt. The `pause()` builtin executes the `pause` instruction, which provides a hint to the processor that the code is in a spin loop, improving power efficiency and performance on hyper-threaded processors.

lowl

```
// Critical section with interrupt management
fn atomic_update(shared_data: ptr_mut<u64>, new_value: u64):
    disable_interrupts()
    *shared_data = new_value
    enable_interrupts()

// Idle loop with power saving
fn idle_loop():
    while true:
        if has_pending_work():
            process_work()
        else:
            halt() // Wait for next interrupt

// Spin loop with pause instruction for better performance
fn spin_wait(condition: &bool):
    while not *condition:
        pause() // Improves hyper-threading efficiency

// Interrupt handler prologue/epilogue (compiler-generated with #
[interrupt])
#[interrupt]
fn timer_isr():
    // The compiler automatically inserts CLI at entry
    // and STI before IRET (or leaves CLI based on handler type)
    update_system_timer()
    // Compiler generates IRETQ at exit
```

15.3 Port I/O Builtins

Port I/O builtins provide access to x86's separate I/O address space using the `in` and `out` instructions. These are essential for communicating with legacy devices, the PIC (Programmable Interrupt Controller), the PIT (Programmable Interval Timer), and other x86 peripherals. The builtins come in three widths: 8-bit (`port_read8` , `port_write8`), 16-bit (`port_read16` , `port_write16`), and 32-bit (`port_read32` , `port_write32`). All port I/O builtins are

serializing, meaning they will not be reordered with respect to other I/O operations, but memory fences may still be needed for ordering with memory accesses.

lowl

```
// PIC initialization (Programmable Interrupt Controller)
fn init_pic():
    // Start initialization sequence
    port_write8(0x20, 0x11) // ICW1: Edge triggered, cascade, ICW4 needed
    port_write8(0xA0, 0x11)

    port_write8(0x21, 0x20) // ICW2: Master IRQ base = 32
    port_write8(0xA1, 0x28) // ICW2: Slave IRQ base = 40

    port_write8(0x21, 0x04) // ICW3: Slave on IRQ2
    port_write8(0xA1, 0x02) // ICW3: Slave ID

    port_write8(0x21, 0x01) // ICW4: 8086 mode
    port_write8(0xA1, 0x01)

    // Mask all interrupts initially
    port_write8(0x21, 0xFF)
    port_write8(0xA1, 0xFF)

// PIT (Programmable Interval Timer) configuration
fn set_timer_frequency(hz: u32):
    let divisor = 1193182 / hz
    port_write8(0x43, 0x36) // Channel 0, lobyte/hibyte, rate generator
    port_write8(0x40, (divisor & 0xFF) as u8)
    port_write8(0x40, ((divisor >> 8) & 0xFF) as u8)

// Reading CMOS RTC
fn read_cmos_time() -> (u8, u8, u8):
    // Wait for CMOS update in progress flag to clear
    while port_read8(0x70) & 0x80 != 0:
        pause()

    port_write8(0x70, 0x00) // Seconds
    let seconds = port_read8(0x71)

    port_write8(0x70, 0x02) // Minutes
    let minutes = port_read8(0x71)

    port_write8(0x70, 0x04) // Hours
    let hours = port_read8(0x71)

    return (hours, minutes, seconds)

// PS/2 keyboard controller access
```

```
fn keyboard_read_scancode() -> u8:
    // Wait for output buffer full
    while (port_read8(0x64) & 1) == 0:
        pause()
    return port_read8(0x60)
```

15.4 Control Register Builtins

Control registers (CR0-CR4) control processor-wide features including paging, protection, and mode selection. lowl provides `read_cr0()`, `read_cr2()`, `read_cr3()`, `read_cr4()` and their corresponding write functions. `read_cr2()` is particularly important for page fault handlers because it contains the faulting linear address. `read_cr3()` returns the current page table root pointer. Writing to CR3 flushes the TLB (Translation Lookaside Buffer). `invlpg(address)` selectively invalidates a single TLB entry, which is more efficient than reloading CR3 when only one page mapping changes.

lowl

```
// Enable paging
fn enable_paging(page_table_root: u64):
    write_cr3(page_table_root)
    let cr0 = read_cr0()
    write_cr0(cr0 | (1 << 31)) // Set PG bit

// Enable PAE (Physical Address Extension) for long mode
fn enable_pae():
    let cr4 = read_cr4()
    write_cr4(cr4 | (1 << 5)) // Set PAE bit

// Page fault handler using CR2
#[interrupt]
fn page_fault_handler():
    let fault_address = read_cr2()
    let error_code = asm("mov rax, [rsp+16]") // Error code on stack

    if fault_address >= USER_SPACE_START:
        handle_user_page_fault(fault_address, error_code)
    else:
        handle_kernel_page_fault(fault_address, error_code)

// Flush TLB for a specific page
fn unmap_page(virtual_address: u64):
    // Remove from page tables
    clear_page_table_entry(virtual_address)
    // Flush TLB to remove stale mapping
    invlpg(virtual_address)

// Get current privilege level
fn get_cpl() -> u8:
```

```
let cs: u16
asm("mov %0, cs" : "=r"(cs))
return (cs & 3) as u8
```

15.5 Model-Specific Register (MSR) Builtins

Model-Specific Registers (MSRs) control processor-specific features including long mode, SYSCALL configuration, performance counters, and thermal management. The `read_msr(msr_index)` and `write_msr(msr_index, value)` builtins access these registers. MSR access requires ring 0 privilege. Important MSRs include: EFER (0xC0000080) for long mode and NXE; STAR (0xC0000081) for SYSCALL segment selectors; LSTAR (0xC0000082) for the SYSCALL entry point; and FS_BASE (0xC0000100) for the FS segment base used in thread-local storage.

lowl

```
// Enable long mode (64-bit)
fn enable_long_mode():
    let efer = read_msr(0xC0000080)
    write_msr(0xC0000080, efer | (1 << 8)) // Set LME bit

// Enable NX (No-Execute) page protection
fn enable_nx():
    let efer = read_msr(0xC0000080)
    write_msr(0xC0000080, efer | (1 << 11)) // Set NXE bit

// Configure SYSCALL entry point for user-to-kernel transitions
fn setup_syscall_entry(entry_point: u64):
    // STAR: bits 47-32 = kernel CS base, bits 63-48 = user CS base
    let user_cs = 0x18 | 3 // User code segment (ring 3)
    let kernel_cs = 0x08 // Kernel code segment
    let star = ((kernel_cs as u64) << 48) | ((user_cs as u64) << 32)
    write_msr(0xC0000081, star)

    // LSTAR: syscall entry point address
    write_msr(0xC0000082, entry_point)

    // SFMASK: flags to clear on syscall (IF, DF, etc.)
    write_msr(0xC0000084, 0x200) // Clear IF (interrupts)

// Set FS segment base for thread-local storage
fn set_fs_base(base: u64):
    write_msr(0xC0000100, base) // FS_BASE

// Read performance counter
fn read_perf_counter(counter_index: u32) -> u64:
    return read_msr(0xC0000000 + (counter_index as u64))

// Configure performance monitoring
```

```
fn setup_perf_counter(counter_index: u32, event_select: u32):
    let config = (event_select as u64) | (1 << 22) // Enable counter
    write_msr(0xC0000100 + (counter_index as u64), config)
```

15.6 Timestamp Counter (TSC) Builtins

The Time Stamp Counter (TSC) is a 64-bit register that counts cycles since processor reset.

`rdtsc()` reads the TSC and returns a 64-bit value. On modern processors with "invariant TSC", the TSC runs at a constant frequency regardless of power management states, making it suitable for high-resolution timing. `rdtscp()` reads the TSC in a serializing manner and also returns the processor ID, which is useful for timing on multi-core systems where TSCs may not be synchronized across cores.

lowl

```
// High-resolution timing measurement
fn measure_time(f: fn()) -> u64:
    let start = rdtsc()
    f()
    let end = rdtsc()
    return end - start

// Calibrate TSC to get frequency (requires known timer)
fn calibrate_tsc() -> u64:
    // Program PIT for 10ms
    let pit_divisor = 1193182 / 100 // 100Hz = 10ms
    port_write8(0x43, 0x36)
    port_write8(0x40, (pit_divisor & 0xFF) as u8)
    port_write8(0x40, ((pit_divisor >> 8) & 0xFF) as u8)

    let start = rdtsc()
    // Wait for 10ms (simplified - would check PIT flags)
    for i in 0..1000000:
        pause()
    let end = rdtsc()

    return (end - start) * 100 // Cycles per second

// Serialized TSC read (avoids out-of-order execution)
fn rdtsc_serialized() -> u64:
    mfence()
    let result = rdtsc()
    lfence()
    return result

// Measure execution time of critical section
fn measure_critical_section() -> u64:
    let start = rdtscp()
```

```
// Critical section
critical_operation()

let end = rdtscp()
return end - start
```

15.7 CPU Identification (CPUID) Builtin

The `cpuid(leaf, subleaf)` builtin executes the CPUID instruction, which returns information about the processor's features, vendor, and capabilities. The function returns a 32-bit value (EAX), but also provides access to EBX, ECX, and EDX through a struct return or separate builtins. CPUID is used to detect SIMD support, hyper-threading, virtualization features, and cache topology.

lowl

```
// Detect CPU vendor string
fn get_cpu_vendor() -> string:
    let eax = cpuid(0, 0) // Maximum leaf
    // cpuid with leaf 0 returns vendor in EBX, EDX, ECX
    let vendor = asm("mov ebx, &vendor") // Simplified
    return vendor

// Check for SSE support
fn has_sse() -> bool:
    let features = cpuid(1, 0) // EDX bits
    return (features & (1 << 25)) != 0

// Check for AVX support
fn has_avx() -> bool:
    let features = cpuid(1, 0) // ECX bits
    return (features & (1 << 28)) != 0

// Check for AVX-512 support
fn has_avx512() -> bool:
    let features = cpuid(7, 0) // EBX bits
    return (features & (1 << 16)) != 0

// Detect hyper-threading
fn has_hyperthreading() -> bool:
    let features = cpuid(1, 0) // EDX bits
    return (features & (1 << 28)) != 0

// Get cache line size
fn get_cache_line_size() -> u32:
    let info = cpuid(0x80000006, 0) // Extended cache info
    return (info >> 8) & 0xFF

// Initialize SIMD features based on CPU detection
```

```
fn init_simd():
    if has_avx512():
        // Enable AVX-512 (set XCR0 bits)
        let xcr0 = read_xcr0()
        write_xcr0(xcr0 | 0xE0) // Enable AVX-512 state
        mxcsr_set(0x1F80) // Mask SIMD exceptions
    elif has_avx():
        let xcr0 = read_xcr0()
        write_xcr0(xcr0 | 0x06) // Enable SSE and AVX
        mxcsr_set(0x1F80)
    elif has_sse():
        // SSE enabled by default in 64-bit mode
        mxcsr_set(0x1F80)
```

15.8 Cache and Memory Management Builtins

Memory management builtins provide control over caching behavior and memory ordering.

`mfence()`, `lfence()`, and `sfence()` execute memory fence instructions that enforce ordering between memory accesses. `prefetch(address, hint)` prefetches cache lines into various cache levels. These builtins are critical for lock-free programming and performance optimization.

lowl

```
// Memory barrier for mutex unlock
fn mutex_unlock(mutex: ptr_mut<u64>):
    mfence() // Ensure all previous writes complete
    asm("mov byte [%0], 1" : : "r"(mutex))

// Load fence for ordering loads before subsequent loads
fn atomic_load_acq(ptr: ptr<u64>) -> u64:
    let value = *ptr
    lfence() // Ensure subsequent loads happen after
    return value

// Store fence for ordering stores
fn atomic_store_rel(ptr: ptr_mut<u64>, value: u64):
    sfence() // Ensure previous stores complete
    *ptr = value

// Prefetch data for performance
fn process_large_array(arr: ptr<f64, len: u64>):
    for i in 0..len:
        // Prefetch 8 cache lines ahead (512 bytes)
        if i + 8 < len:
            let prefetch_addr = &arr[i + 8]
            prefetch(prefetch_addr, 0) // Prefetch into L1
        process(arr[i])
```

```
// Non-temporal store (bypasses cache) for large writes
fn memset_non_temporal(dest: ptr<u8>, value: u8, count: u64):
    // Write in 64-byte chunks using MOVNTDQ
    let vec_val = vec16_u8.broadcast(value)
    for i in 0..(count / 64):
        vec_val.store_nt(dest + i * 64)
    sfence() // Ensure writes complete before continuing
```

15.9 FPU and SIMD Control Builtins

The floating-point unit (FPU) and SIMD registers require initialization and sometimes explicit state management. `fpu_init()` executes `finit` to initialize the FPU. `fpu_save(buffer)` and `fpu_restore(buffer)` use `fxsave` and `fxrstor` to save and restore FPU/SIMD state. `mxcsr_get()` and `mxcsr_set(flags)` read and write the MXCSR register, which controls SIMD exception masking and rounding modes.

lowl

```
// Initialize FPU and SIMD
fn init_fpu():
    fpu_init() // Initialize FPU
    let mxcsr = mxcsr_get()
    mxcsr_set(mxcsr | 0x1F80) // Mask all SIMD exceptions

// Save FPU state for context switching
fn save_fpu_state(state_buffer: ptr<u8>):
    fpu_save(state_buffer)

// Restore FPU state
fn restore_fpu_state(state_buffer: ptr<u8>):
    fpu_restore(state_buffer)

// Set rounding mode for floating-point operations
fn set_rounding_mode(mode: RoundingMode):
    let mxcsr = mxcsr_get()
    // Clear rounding control bits (13-14)
    let mxcsr_cleared = mxcsr & ~(3 << 13)
    // Set new rounding mode
    let new_mxcsr = mxcsr_cleared | ((mode as u32) << 13)
    mxcsr_set(new_mxcsr)

// Set flush-to-zero mode (denormals become zero)
fn set_flush_to_zero(enabled: bool):
    let mxcsr = mxcsr_get()
    if enabled:
        mxcsr_set(mxcsr | (1 << 15))
    else:
        mxcsr_set(mxcsr & ~(1 << 15))
```

15.10 Atomic Memory Operations Builtins

lowl provides atomic operations that compile to `lock`-prefixed instructions. These are essential for lock-free programming. The builtins include `atomic_increment`, `atomic_decrement`, `atomic_add`, `atomic_cas` (compare-and-swap), `atomic_xchg` (exchange), `atomic_load`, `atomic_store`, and `atomic_flag_test_and_set`. All atomic builtins include appropriate memory barriers for the specified ordering.

lowl

```
// Simple reference counter using atomic operations
struct RefCount:
    count: u64

impl RefCount:
    fn new() -> RefCount:
        return RefCount{count: 1}

    fn increment():
        atomic_increment(&this.count)

    fn decrement() -> bool:
        let old = atomic_decrement(&this.count)
        return old == 0

// Lock-free stack using compare-and-swap
template<class T>
class LockFreeStack:
    private:
        struct Node:
            value: T
            next: ptr<Node>

        head: ptr<Node>

    public:
        fn push(value: T):
            let node = physical_alloc(sizeof(Node), alignof(Node)) as
ptr<Node>
            node.value = value

            while true:
                let old_head = this.head
                node.next = old_head
                if atomic_cas(&this.head, old_head, node):
                    break
                pause()

        fn pop() -> Option<T>:
            while true:
```

```

        let old_head = this.head
        if old_head == null:
            return Option.none()
        let new_head = old_head.next
        if atomic_cas(&this.head, old_head, new_head):
            let value = old_head.value
            physical_free(old_head)
            return Option.some(value)
        pause()

// Sequential counter with atomic operations
let global_counter: u64 = 0

fn get_next_id() -> u64:
    return atomic_add(&global_counter, 1)

// Double-checked locking pattern
fn get_singleton() -> ptr<Singleton>:
    static instance: ptr<Singleton> = null
    if instance == null:
        disable_interrupts()
        if instance == null:
            instance = Singleton.new()
        enable_interrupts()
    return instance

```

15.11 Complete Chapter Example: System Information Tool

This example demonstrates most system programming builtins in a comprehensive system information tool that reports CPU features, memory configuration, timer frequencies, and performs benchmarks.

lowl

```

// sysinfo.lowl - System Information Tool using lowl builtins
// Compile: lowlc sysinfo.lowl -o sysinfo.asm -O2

//
=====
// CPU FEATURE DETECTION
//
=====

struct CPUFeatures:
    vendor: string
    model: u32
    family: u32
    stepping: u32
    has_sse: bool

```

```

has_sse2: bool
has_sse3: bool
has_sse41: bool
has_sse42: bool
has_avx: bool
has_avx2: bool
has_avx512f: bool
has_avx512bw: bool
has_avx512dq: bool
has_avx512vl: bool
has_hypervisor: bool
physical_cores: u32
logical_cores: u32

```

```

fn detect_cpu_features() -> CPUFeatures:
    let mut features = CPUFeatures{}

    // Get vendor string
    let leaf0 = cpuid(0, 0)
    features.vendor = get_vendor_string()

    // Get processor info from leaf 1
    let leaf1 = cpuid(1, 0)
    features.stepping = leaf1 & 0xF
    features.model = (leaf1 >> 4) & 0xF
    features.family = (leaf1 >> 8) & 0xF
    features.has_sse = (cpuid(1, 1) & (1 << 25)) != 0
    features.has_sse2 = (cpuid(1, 1) & (1 << 26)) != 0
    features.has_sse3 = (cpuid(1, 1) & (1 << 0)) != 0
    features.has_sse41 = (cpuid(1, 1) & (1 << 19)) != 0
    features.has_sse42 = (cpuid(1, 1) & (1 << 20)) != 0
    features.has_avx = (cpuid(1, 1) & (1 << 28)) != 0
    features.has_hypervisor = (cpuid(1, 1) & (1 << 31)) != 0

    // Extended features from leaf 7
    let leaf7 = cpuid(7, 0)
    features.has_avx2 = (leaf7 & (1 << 5)) != 0
    features.has_avx512f = (leaf7 & (1 << 16)) != 0
    features.has_avx512bw = (leaf7 & (1 << 30)) != 0
    features.has_avx512dq = (leaf7 & (1 << 17)) != 0
    features.has_avx512vl = (leaf7 & (1 << 31)) != 0

    // Get core counts
    if features.has_hypervisor:
        features.physical_cores = 1
        features.logical_cores = 1
    else:
        features.physical_cores = (leaf7 >> 16) & 0xFF
        features.logical_cores = (leaf7 >> 8) & 0xFF

```

```

    return features

//
=====

// MEMORY INFORMATION
//
=====

struct MemoryInfo:
    total_ram: u64
    usable_ram: u64
    page_size: u64
    cache_line_size: u32

fn detect_memory() -> MemoryInfo:
    let mut info = MemoryInfo{}
    info.page_size = 4096
    info.cache_line_size = get_cache_line_size()

    // Read CR3 to get page table root
    let cr3 = read_cr3()

    // Memory size from e820 or Multiboot would go here
    // For demo, we use fixed values
    info.total_ram = 1024 * 1024 * 1024 // 1GB
    info.usable_ram = 960 * 1024 * 1024 // 960MB

    return info

//
=====

// TIMER BENCHMARKS
//
=====

fn benchmark_tsc() -> u64:
    // Measure TSC frequency using PIT calibration
    // Program PIT for 10ms
    const PIT_FREQ: u64 = 1193182
    const TARGET_MS: u64 = 10
    let divisor = PIT_FREQ / (1000 / TARGET_MS)

    port_write8(0x43, 0x36)
    port_write8(0x40, (divisor & 0xFF) as u8)
    port_write8(0x40, ((divisor >> 8) & 0xFF) as u8)

    let start = rdtsc()

```

```

// Wait for PIT to reach zero (simplified - would check status)
for i in 0..1000000:
    pause()

let end = rdtsc()
let cycles_per_10ms = end - start
let cycles_per_sec = cycles_per_10ms * 100

return cycles_per_sec

//
=====
// SYSTEM INFORMATION DISPLAY
//
=====

fn print_system_info():
    print_string("\n")
    print_string("=" * 60)
    print_string("\n")
    print_string("LOWL SYSTEM INFORMATION TOOL\n")
    print_string("Using Hardware Builtins\n")
    print_string("=" * 60)
    print_string("\n\n")

    // CPU Information
    print_string("CPU INFORMATION\n")
    print_string("-----\n")

    let cpu = detect_cpu_features()
    print_string("Vendor: ")
    print_string(cpu.vendor)
    print_string("\n")
    print_string("Family: ")
    print_dec(cpu.family)
    print_string(" Model: ")
    print_dec(cpu.model)
    print_string(" Stepping: ")
    print_dec(cpu.stepping)
    print_string("\n")
    print_string("Physical cores: ")
    print_dec(cpu.physical_cores)
    print_string(" Logical cores: ")
    print_dec(cpu.logical_cores)
    print_string("\n\n")

    print_string("SIMD Support:\n")
    if cpu.has_sse: print_string(" SSE")
    if cpu.has_sse2: print_string(" SSE2")

```

```
if cpu.has_sse3: print_string(" SSE3")
if cpu.has_sse41: print_string(" SSE4.1")
if cpu.has_sse42: print_string(" SSE4.2")
print_string("\n")
if cpu.has_avx: print_string(" AVX")
if cpu.has_avx2: print_string(" AVX2")
print_string("\n")
if cpu.has_avx512f: print_string(" AVX-512F")
if cpu.has_avx512bw: print_string(" AVX-512BW")
if cpu.has_avx512dq: print_string(" AVX-512DQ")
if cpu.has_avx512vl: print_string(" AVX-512VL")
print_string("\n\n")

if cpu.has_hypervisor:
    print_string("Running under hypervisor\n\n")

// Memory Information
print_string("MEMORY INFORMATION\n")
print_string("-----\n")

let mem = detect_memory()
print_string("Page size: ")
print_dec(mem.page_size)
print_string(" bytes\n")
print_string("Cache line size: ")
print_dec(mem.cache_line_size)
print_string(" bytes\n")
print_string("Total RAM: ")
print_dec(mem.total_ram / (1024 * 1024))
print_string(" MB\n")
print_string("Usable RAM: ")
print_dec(mem.usable_ram / (1024 * 1024))
print_string(" MB\n\n")

// Control Register Information
print_string("CONTROL REGISTERS\n")
print_string("-----\n")

let cr0 = read_cr0()
let cr2 = read_cr2()
let cr3 = read_cr3()
let cr4 = read_cr4()

print_string("CR0: 0x")
print_hex(cr0, 16)
print_string("\n")
print_string("CR2: 0x")
print_hex(cr2, 16)
print_string("\n")
```

```

print_string("CR3: 0x")
print_hex(cr3, 16)
print_string("\n")
print_string("CR4: 0x")
print_hex(cr4, 16)
print_string("\n\n")

// Timer Benchmark
print_string("TIMER BENCHMARK\n")
print_string("-----\n")

let tsc_freq = benchmark_tsc()
print_string("TSC Frequency: ")
print_dec(tsc_freq)
print_string(" Hz")
if tsc_freq >= 1_000_000_000:
    print_string(" (")
    print_dec(tsc_freq / 1_000_000_000)
    print_string(" GHz)")
elif tsc_freq >= 1_000_000:
    print_string(" (")
    print_dec(tsc_freq / 1_000_000)
    print_string(" MHz)")
print_string("\n\n")

// Performance measurement
print_string("PERFORMANCE MEASUREMENT\n")
print_string("-----\n")

// Measure simple operation
let start = rdtsc()
for i in 0..1000:
    let x = i * i
    pause()
let end = rdtsc()
let cycles_per_op = (end - start) / 1000

print_string("Cycles per simple operation: ")
print_dec(cycles_per_op)
print_string("\n")

//
=====

// MAIN
//
=====

fn main() -> u32:
    // Initialize FPU and SIMD

```

```

init_fpu()

// Print system information
print_system_info()

// Demonstrate atomic operations
print_string("\nATOMIC OPERATIONS DEMO\n")
print_string("-----\n")

let counter: u64 = 0
let iterations: u64 = 100000

let start = rdtsc()
for i in 0..iterations:
    atomic_increment(&counter)
let end = rdtsc()

print_string("Atomic increments: ")
print_dec(iterations)
print_string(" in ")
print_dec(end - start)
print_string(" cycles\n")
print_string("Final counter value: ")
print_dec(counter)
print_string("\n")

return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:

```

```

        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut temp = value
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

fn print_hex(value: u64, width: u64):
    let hex_digits = "0123456789ABCDEF"
    for i in (width - 1)..0 step -1:
        let shift = i * 4
        let nibble = (value >> shift) & 0xF
        print_char(hex_digits[nibble as u64] as u8)

fn get_vendor_string() -> string:
    static vendor: array<u8, 13> = [0; 13]
    // Simplified - actual cpuid would need inline assembly
    asm("mov eax, 0; cpuid; mov [%0], ebx; mov [%0+4], edx; mov [%0+8], ecx"
: : "r"(&vendor))
    return &vendor[0] as string

// Repeat string operator
operator*(s: string, count: u64) -> string:
    let mut result = ""
    for i in 0..count:
        result = result + s
    return result

```

Expected Output:

text

```

=====
LOWL SYSTEM INFORMATION TOOL
Using Hardware Builtins
=====

CPU INFORMATION
-----
Vendor: GenuineIntel

```

Family: 6 Model: 158 Stepping: 10
Physical cores: 6 Logical cores: 12

SIMD Support:

SSE SSE2 SSE3 SSE4.1 SSE4.2
AVX AVX2
AVX-512F AVX-512BW AVX-512DQ AVX-512VL

MEMORY INFORMATION

Page size: 4096 bytes
Cache line size: 64 bytes
Total RAM: 1024 MB
Usable RAM: 960 MB

CONTROL REGISTERS

CR0: 0x80000011
CR2: 0x00000000
CR3: 0x00001000
CR4: 0x00000620

TIMER BENCHMARK

TSC Frequency: 2500000000 Hz (2.5 GHz)

PERFORMANCE MEASUREMENT

Cycles per simple operation: 3

ATOMIC OPERATIONS DEMO

Atomic increments: 100000 in 4250000 cycles
Final counter value: 100000

This concludes **Chapter 15: System Programming Builtins**. The chapter covered interrupt control (disable/enable/halt/pause), port I/O (read/write at 8/16/32-bit widths), control register access (CR0-CR4, invlpg), MSR access (rdmsr/wrmsr), timestamp counter (rdtsc/rdtscp), CPUID for feature detection, cache management (mfence/lfence/sfence/prefetch), FPU/SIMD control, and atomic operations. These builtins provide complete hardware access without inline assembly, enabling the development of kernels, drivers, and high-performance systems code entirely in lowl.

Chapter 16: SIMD Vector Operations

16.1 Introduction to SIMD in lowl

Single Instruction, Multiple Data (SIMD) is a parallel computing technique where a single instruction operates on multiple data elements simultaneously. Modern x86_64 processors support several SIMD instruction set extensions: SSE (Streaming SIMD Extensions) with 128-bit registers,

AVX (Advanced Vector Extensions) with 256-bit registers, and AVX-512 with 512-bit registers. lowl provides first-class SIMD vector types that map directly to these hardware registers. Unlike languages that require compiler intrinsics or auto-vectorization heuristics, lowl's SIMD types are built into the language, allowing you to write explicit vector code that compiles to optimal assembly. The compiler automatically handles register allocation, instruction selection, and alignment requirements, while you focus on the parallel algorithm. This chapter covers all SIMD vector types, operations, memory access patterns, and optimization techniques for high-performance computing.

16.2 SIMD Vector Types Overview

lowl provides vector types for each SIMD level and data type combination. For single-precision floats (f32), the types are `vec4_f32` (SSE, 4 elements), `vec8_f32` (AVX, 8 elements), and `vec16_f32` (AVX-512, 16 elements). For double-precision floats (f64), the types are `vec2_f64` (SSE2, 2 elements), `vec4_f64` (AVX, 4 elements), and `vec8_f64` (AVX-512, 8 elements). For integers, lowl provides vector types for 8, 16, 32, and 64-bit integers at each SIMD level, though this chapter focuses on floating-point vectors as they are most common in high-performance computing. Mask types (`mask8`, `mask16`, `mask64`) are used with AVX-512 for predicated operations, allowing selective processing of individual vector lanes.

lowl

```
// SSE vector types (128-bit registers)
let sse_f32: vec4_f32 = vec4_f32(1.0, 2.0, 3.0, 4.0)
let sse_f64: vec2_f64 = vec2_f64(1.5, 2.5)
let sse_i32: vec4_i32 = vec4_i32(1, 2, 3, 4)
let sse_i64: vec2_i64 = vec2_i64(100, 200)

// AVX vector types (256-bit registers)
let avx_f32: vec8_f32 = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)
let avx_f64: vec4_f64 = vec4_f64(1.0, 2.0, 3.0, 4.0)

// AVX-512 vector types (512-bit registers)
let avx512_f32: vec16_f32 = vec16_f32(
    1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
    9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0
)
let avx512_f64: vec8_f64 = vec8_f64(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)

// Mask types for AVX-512 predication
let mask_all_true: mask16 = 0xFFFF
let mask_alternating: mask16 = 0xAAAA
let mask_lower_half: mask16 = 0x00FF
```

16.3 Vector Construction and Initialization

Vectors can be constructed in several ways: from individual elements (explicit constructor), from memory (load operations), using broadcast (replicate a scalar), using set operations (set individual lanes), or using sequences (iota for arithmetic progressions). The compiler generates the optimal

instruction sequence for each construction method, using immediate values when possible and register-to-register moves when needed.

lowl

```
// Explicit element construction
let explicit = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)

// Load from memory (must be aligned for best performance)
let aligned_data: array<f32, 8> = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]
let loaded = vec8_f32.load(&aligned_data[0])

// Unaligned load (slightly slower on some CPUs)
let unaligned_data: array<f32, 9> = [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0]
let unaligned = vec8_f32.loadu(&unaligned_data[1])

// Broadcast scalar to all lanes (efficient, uses vbroadcastss)
let broadcast = vec8_f32.broadcast(3.14)

// Set single lane (preserves other lanes)
let mut vec = vec8_f32(0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0)
vec.set(3, 42.0) // Fourth lane becomes 42.0

// Iota (arithmetic progression)
let iota = vec8_f32.iota(0.0, 1.0) // [0, 1, 2, 3, 4, 5, 6, 7]

// Zero vector (all zeros, fast using vxorps)
let zero = vec8_f32.zero()

// One vector (all ones)
let one = vec8_f32.one()

// From integer (conversion)
let ints: vec8_i32 = vec8_i32(1, 2, 3, 4, 5, 6, 7, 8)
let floats = ints.f32_convert()
```

16.4 Arithmetic Operations

Vector arithmetic operations are element-wise: each lane of the result is computed from the corresponding lanes of the operands. For binary operations, both vectors must have the same number of elements. For scalar operations, the scalar is broadcast to all lanes before the operation. The compiler generates the appropriate SIMD instruction: `addps` for SSE addition, `vaddps` for AVX, `vmulps` for multiplication, `vdivps` for division, `vfmadd213ps` for fused multiply-add, and so on. Fused multiply-add (FMA) is particularly important because it combines multiplication and addition into a single instruction with higher precision and lower latency.

lowl

```
// Basic arithmetic (element-wise)
let a = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)
let b = vec8_f32(8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0)

let sum = a + b           // [9, 9, 9, 9, 9, 9, 9, 9]
let diff = a - b          // [-7, -5, -3, -1, 1, 3, 5, 7]
let prod = a * b          // [8, 14, 18, 20, 20, 18, 14, 8]
let quot = a / b          // [0.125, 0.286, 0.5, 0.8, 1.25, 2.0, 3.5, 8.0]

// Scalar operations (broadcast scalar to all lanes)
let scaled = a * 2.0      // [2, 4, 6, 8, 10, 12, 14, 16]
let shifted = a + 10.0    // [11, 12, 13, 14, 15, 16, 17, 18]

// Fused Multiply-Add (FMA): result = (a * b) + c
// Single instruction with no intermediate rounding
let a_fma = vec8_f32(2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 2.0)
let b_fma = vec8_f32(3.0, 3.0, 3.0, 3.0, 3.0, 3.0, 3.0, 3.0)
let c_fma = vec8_f32(1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0)
let fma_result = a_fma.fma(&b_fma, &c_fma) // [7, 7, 7, 7, 7, 7, 7, 7]

// Fused Multiply-Subtract: result = (a * b) - c
let fms_result = a_fma.fms(&b_fma, &c_fma) // [5, 5, 5, 5, 5, 5, 5, 5]

// Negation
let neg = -a              // [-1, -2, -3, -4, -5, -6, -7, -8]

// Absolute value
let abs_vals = a.abs()    // No change for positive values
let negative = vec8_f32(-1.0, -2.0, -3.0, -4.0, -5.0, -6.0, -7.0, -8.0)
let abs_neg = negative.abs() // [1, 2, 3, 4, 5, 6, 7, 8]
```

16.5 Horizontal Operations

Horizontal operations combine elements within a single vector, as opposed to vertical operations that combine corresponding elements from multiple vectors. Horizontal operations are more expensive than vertical operations because they require shuffling data within the register.

`hadd()` (horizontal add) sums adjacent pairs and produces a new vector. `hadd_all()` sums all elements. Similarly, `hmax()`, `hmin()`, `hmul()`, and `hsum()` provide various reductions. For AVX-512, these operations can use mask registers to selectively include or exclude elements from the reduction.

lowl

```
// Horizontal add (adjacent pairs)
let v = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)
let hadd = v.hadd() // After first hadd: [1+2, 3+4, 5+6, 7+8] = [3, 7, 11, 15] in 4 lanes
// Second hadd would produce [3+7, 11+15] = [10, 26]
// Third hadd produces [10+26] = [36] (scalar)
```

```
// Horizontal sum (all elements)
let sum_all = v.hadd_all() // 36.0

// Horizontal maximum
let max_all = v.hmax() // 8.0

// Horizontal minimum
let min_all = v.hmin() // 1.0

// Horizontal product (all elements)
let prod_all = v.hmul() // 40320.0 (1*2*3*4*5*6*7*8)

// Dot product (sum of element-wise products)
let a_dot = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)
let b_dot = vec8_f32(8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0)
let dot = a_dot.dot(&b_dot) // 1*8 + 2*7 + 3*6 + 4*5 + 5*4 + 6*3 + 7*2 + 8*1 = 120

// Efficient dot product using FMA (avx512)
let dot_fma = a_dot.dot_fma(&b_dot) // Same result but using FMA for precision

// Horizontal operations with masks (AVX-512)
let mask: mask8 = 0b10101010 // Even lanes only
let masked_hsum = v.hsum_mask(mask) // Sum of elements where mask bit is 1
```

16.6 Comparison Operations

Comparison operations between vectors produce mask registers (for AVX-512) or vectors of all-ones (0xFFFFFFFF) for true and zeros for false. For SSE and AVX, comparison results are vectors that can be used with blend operations. For AVX-512, comparison results are mask registers (k0-k7) that can be used for predicated operations. The available comparators include equal, not equal, less than, less than or equal, greater than, greater than or equal, and ordered/unordered (for NaN handling).

lowl

```
// Vector comparison (produces mask for AVX-512)
let a_cmp = vec16_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0)
let b_cmp = vec16_f32(5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0)

let eq_mask = a_cmp.cmpeq(b_cmp) // Bits where a[i] == 5.0
let lt_mask = a_cmp.cmlt(b_cmp) // Bits where a[i] < 5.0
let le_mask = a_cmp.cmples(b_cmp) // Bits where a[i] <= 5.0
let gt_mask = a_cmp.cmpgt(b_cmp) // Bits where a[i] > 5.0
let ge_mask = a_cmp.cmpge(b_cmp) // Bits where a[i] >= 5.0
```

```

let ne_mask = a_cmp.cmpneq(b_cmp)    // Bits where a[i] != 5.0

// NaN-aware comparisons
let nan = vec16_f32.broadcast(0.0/0.0) // NaN
let ordered_mask = a_cmp.cmpord(b_cmp)  // Neither operand is NaN
let unordered_mask = a_cmp.cmpunord(b_cmp) // At least one operand is NaN

// Using masks for predicated operations (AVX-512)
let data = vec16_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0,
11.0, 12.0, 13.0, 14.0, 15.0, 16.0)
let mask_lt_10 = data.cmlt(vec16_f32.broadcast(10.0))

// Add 100 only to elements less than 10
let result = data.masked_add(mask_lt_10, vec16_f32.broadcast(100.0))
// Elements 0-8 become 101-108? Actually 1+100=101, etc. Elements 9-15
unchanged

```

16.7 Permute and Shuffle Operations

Permute and shuffle operations rearrange elements within a vector or between two vectors. These operations are essential for data reorganization, transposition, and certain algorithms like FFT.

permute allows arbitrary reordering of elements within a vector using a control mask. **shuffle** combines elements from two vectors. **blend** selects elements from two vectors based on a mask. **extract** and **insert** operations move elements between vectors and scalar registers.

lowl

```

// Permute (reorder elements within a single vector)
let v_perm = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)

// Permute using immediate control (compile-time constant)
// 0b11010010 = [2,0,3,1] for 4-element permute
let permuted = v_perm.permute(0b11010010) // [3, 1, 4, 2, ...] for first 4
lanes

// Permute using vector control (runtime-determined)
let control = vec8_i32(3, 2, 1, 0, 7, 6, 5, 4)
let permuted_runtime = v_perm.permutevar(control)

// Shuffle (combine elements from two vectors)
let a_shuf = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)
let b_shuf = vec8_f32(9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0)

// Shuffle between vectors (takes elements from both sources)
let shuffled = a_shuf.shuffle(b_shuf, 0b01000111) // Complex mask,
compiler-specific

// Blend (select from two vectors based on mask)
let mask_blend: mask8 = 0b10101010

```

```

let blended = a_shuf.blend(b_shuf, mask_blend) // Even lanes from a, odd
from b

// Extract lane to scalar
let fifth = v_perm.extract(4) // 5.0

// Insert scalar into vector
let mut v_mod = v_perm
v_mod = v_mod.insert(4, 99.0) // [1,2,3,4,99,6,7,8]

// Broadcast lane to entire vector
let broadcast_lane = v_perm.broadcast(2) // [3,3,3,3,3,3,3,3]

// Extract high and low halves (for dealing with 256-bit on 512-bit
hardware)
let low = avx512_f32.extract_low() // vec8_f32 (first 8 elements)
let high = avx512_f32.extract_high() // vec8_f32 (last 8 elements)

```

16.8 Load and Store Operations

Memory access patterns are critical for SIMD performance. The compiler generates aligned loads (`v movaps`) when it can guarantee alignment, otherwise unaligned loads (`vmovups`). Non-temporal stores (`stream` operations) bypass the cache, which is useful for writing large amounts of data that won't be read back soon. Gather and scatter operations (AVX-512 only) load and store elements at arbitrary indices, enabling strided access patterns and sparse data processing.

lowl

```

// Aligned load (fastest, requires 16/32/64-byte alignment)
#[align(64)]
let aligned_buffer: array<f32, 16> = [0.0; 16]
let aligned_vec = vec16_f32.load(&aligned_buffer[0])

// Unaligned load (slightly slower, works with any address)
let unaligned_buffer: array<f32, 16> = [0.0; 16]
let unaligned_vec = vec16_f32.loadu(&unaligned_buffer[1])

// Store to memory (aligned)
let mut dest_aligned: array<f32, 16> = [0.0; 16]
vec16_f32.store(&mut dest_aligned[0], result_vec)

// Non-temporal store (bypasses cache, good for streaming writes)
let mut dest_stream: array<f32, 1024> = [0.0; 1024]
vec16_f32.stream(&mut dest_stream[0], result_vec)

// Gather (load from non-contiguous addresses) - AVX-512 only
let base_ptr: ptr<f32> = &data_buffer[0]
let indices = vec8_i64(0, 2, 4, 6, 8, 10, 12, 14)
let gathered = vec8_f32.gather(base_ptr, indices)

```

```
// Scatter (store to non-contiguous addresses) - AVX-512 only
let values = vec8_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0)
let scatter_indices = vec8_i64(0, 2, 4, 6, 8, 10, 12, 14)
values.scatter(base_ptr, scatter_indices)

// Masked gather (only load selected elements)
let gather_mask: mask8 = 0b10101010
let masked_gathered = vec8_f32.gather_mask(base_ptr, indices, gather_mask)

// Compress (remove masked elements)
let compress_vals = vec16_f32(1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0,
10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0)
let compress_mask: mask16 = 0b0101010101010101 // Odd indices only
let compressed = compress_vals.compress(compress_mask) //
[2,4,6,8,10,12,14,16]

// Expand (insert elements according to mask)
let expanded = compressed.expand(compress_mask, vec16_f32.zero())
```

16.9 Mathematical Functions

lowl provides SIMD-accelerated mathematical functions for vectors, including square root, reciprocal, reciprocal square root, and approximate versions for faster but less accurate computation. The approximate versions (`rsqrt_approx` , `rcp_approx`) are useful in graphics and machine learning where small errors are acceptable but performance is critical.

lowl

```
// Square root (element-wise)
let v_sqrt = vec8_f32(1.0, 4.0, 9.0, 16.0, 25.0, 36.0, 49.0, 64.0)
let roots = v_sqrt.sqrt() // [1, 2, 3, 4, 5, 6, 7, 8]

// Reciprocal (1/x)
let v_rcp = vec8_f32(1.0, 2.0, 4.0, 8.0, 16.0, 32.0, 64.0, 128.0)
let reciprocals = v_rcp.rcp() // [1, 0.5, 0.25, 0.125, 0.0625, 0.03125,
0.015625, 0.0078125]

// Reciprocal square root (1/sqrt(x)) - useful for normalization
let v_rsqrt = vec8_f32(1.0, 4.0, 9.0, 16.0, 25.0, 36.0, 49.0, 64.0)
let inv_sqrt = v_rsqrt.rsqrt() // [1, 0.5, 0.333, 0.25, 0.2, 0.1667,
0.1429, 0.125]

// Fast approximate versions (higher throughput, lower precision)
let approx_inv_sqrt = v_rsqrt.rsqrt_approx() // ~12-bit precision, but
faster
let approx_rcp = v_rcp.rcp_approx()

// Vector normalization using SIMD
```

```
fn normalize_vector3(v: vec4_f32) -> vec4_f32: // w component is 0 for
vectors
    let length_sq = v * v
    let length_sq_sum = length_sq.hadd_all()
    let inv_length = vec4_f32.broadcast(length_sq_sum).rsqrt()
    return v * inv_length
```

16.10 Complete Chapter Example: Matrix Multiplication with SIMD

This example demonstrates a complete matrix multiplication implementation using all three SIMD levels (SSE, AVX, AVX-512) with performance benchmarking.

lowl

```
// matmul_simd.lowl - Matrix Multiplication with SSE/AVX/AVX-512
// Compile: lowlc matmul_simd.lowl -o matmul.asm -O3 -f elf

//
=====
// MATRIX TYPES
//
=====

const MATRIX_SIZE: u64 = 1024
const BLOCK_SIZE: u64 = 64

#[align(64)]
struct Matrix:
    rows: u64
    cols: u64
    data: ptr<f32>

impl Matrix:
    fn new(rows: u64, cols: u64) -> Matrix:
        let bytes = rows * cols * sizeof(f32)
        let data = physical_alloc(bytes, 64) as ptr<f32>
        zero_memory(data, bytes)
        return Matrix{rows, cols, data}

    fn random_fill():
        for i in 0..this.rows * this.cols:
            this.data[i] = (i as f32) * 0.001

    fn get(row: u64, col: u64) -> f32:
        return this.data[row * this.cols + col]

    fn set(row: u64, col: u64, value: f32):
        this.data[row * this.cols + col] = value
```

```
//
=====
// SSE IMPLEMENTATION (4-wide)
//
=====

fn matmul_sse(A: &Matrix, B: &Matrix, C: &Matrix):
    let n = A.rows
    let m = A.cols
    let p = B.cols

    for i in 0..n:
        for k in 0..m:
            let aik = A.get(i, k)
            let aik_vec = vec4_f32.broadcast(aik)
            let mut j: u64 = 0
            while j + 4 <= p:
                // Load 4 floats from B[k][j..j+3]
                let b_vec = vec4_f32.load(&B.data[k * p + j])
                // Load current C[i][j..j+3]
                let c_vec = vec4_f32.load(&C.data[i * p + j])
                // Compute and store
                let result = c_vec + aik_vec * b_vec
                result.store(&mut C.data[i * p + j])
                j = j + 4
            // Handle remainder with scalar
            while j < p:
                let c_new = C.get(i, j) + aik * B.get(k, j)
                C.set(i, j, c_new)
                j = j + 1

//
=====
// AVX IMPLEMENTATION (8-wide)
//
=====

fn matmul_avx(A: &Matrix, B: &Matrix, C: &Matrix):
    let n = A.rows
    let m = A.cols
    let p = B.cols

    for i in 0..n:
        for k in 0..m:
            let aik = A.get(i, k)
            let aik_vec = vec8_f32.broadcast(aik)
            let mut j: u64 = 0
            while j + 8 <= p:
```

```

        let b_vec = vec8_f32.load(&B.data[k * p + j])
        let c_vec = vec8_f32.load(&C.data[i * p + j])
        let result = c_vec + aik_vec * b_vec
        result.store(&mut C.data[i * p + j])
        j = j + 8
    while j < p:
        let c_new = C.get(i, j) + aik * B.get(k, j)
        C.set(i, j, c_new)
        j = j + 1

//
=====
// AVX-512 IMPLEMENTATION (16-wide)
//
=====

fn matmul_avx512(A: &Matrix, B: &Matrix, C: &Matrix):
    let n = A.rows
    let m = A.cols
    let p = B.cols

    for i in 0..n:
        for k in 0..m:
            let aik = A.get(i, k)
            let aik_vec = vec16_f32.broadcast(aik)
            let mut j: u64 = 0
            while j + 16 <= p:
                let b_vec = vec16_f32.load(&B.data[k * p + j])
                let c_vec = vec16_f32.load(&C.data[i * p + j])
                let result = c_vec.fma(&aik_vec, &b_vec) // FMA: (aik *
b_vec) + c_vec
                result.store(&mut C.data[i * p + j])
                j = j + 16
            while j < p:
                let c_new = C.get(i, j) + aik * B.get(k, j)
                C.set(i, j, c_new)
                j = j + 1

//
=====
// TILED AVX-512 IMPLEMENTATION (Cache-optimized)
//
=====

fn matmul_avx512_tiled(A: &Matrix, B: &Matrix, C: &Matrix):
    let n = A.rows
    let m = A.cols
    let p = B.cols

```

```

// Process in tiles that fit in L2 cache
for ii in 0..n step BLOCK_SIZE:
  for jj in 0..p step BLOCK_SIZE:
    for kk in 0..m step BLOCK_SIZE:
      let i_max = min(ii + BLOCK_SIZE, n)
      let j_max = min(jj + BLOCK_SIZE, p)
      let k_max = min(kk + BLOCK_SIZE, m)

      for i in ii..i_max:
        for k in kk..k_max:
          let aik = A.get(i, k)
          let aik_vec = vec16_f32.broadcast(aik)
          let mut j = jj
          while j + 16 <= j_max:
            let b_vec = vec16_f32.load(&B.data[k * p + j])
            let c_vec = vec16_f32.load(&C.data[i * p + j])
            let result = c_vec.fma(&aik_vec, &b_vec)
            result.store(&mut C.data[i * p + j])
            j = j + 16
          // Remainder
          while j < j_max:
            let c_new = C.get(i, j) + aik * B.get(k, j)
            C.set(i, j, c_new)
            j = j + 1

//
=====
// BENCHMARK
//
=====

fn benchmark():
  print_string("Matrix Multiplication SIMD Benchmark\n")
  print_string("=====\n\n")

  let size: u64 = 512 // 512x512 matrices (smaller for faster demo)

  print_string("Initializing matrices...\n")
  let A = Matrix.new(size, size)
  let B = Matrix.new(size, size)
  let C1 = Matrix.new(size, size)
  let C2 = Matrix.new(size, size)
  let C3 = Matrix.new(size, size)
  let C4 = Matrix.new(size, size)

  A.random_fill()
  B.random_fill()

  // Warm-up (load into cache)

```

```
matmul_sse(&A, &B, &C1)

// Benchmark SSE
let start = rdtsc()
matmul_sse(&A, &B, &C1)
let end = rdtsc()
let cycles_sse = end - start

// Clear C and warm-up
zero_memory(C2.data, size * size * sizeof(f32))
matmul_avx(&A, &B, &C2)

// Benchmark AVX
start = rdtsc()
matmul_avx(&A, &B, &C2)
end = rdtsc()
let cycles_avx = end - start

// Clear C and warm-up
zero_memory(C3.data, size * size * sizeof(f32))
matmul_avx512(&A, &B, &C3)

// Benchmark AVX-512
start = rdtsc()
matmul_avx512(&A, &B, &C3)
end = rdtsc()
let cycles_avx512 = end - start

// Clear C and warm-up
zero_memory(C4.data, size * size * sizeof(f32))
matmul_avx512_tiled(&A, &B, &C4)

// Benchmark Tiled AVX-512
start = rdtsc()
matmul_avx512_tiled(&A, &B, &C4)
end = rdtsc()
let cycles_tiled = end - start

// Print results
print_string("Matrix size: ")
print_dec(size)
print_string("x")
print_dec(size)
print_string("\n\n")

print_string("SSE (4-wide):      ")
print_dec(cycles_sse)
print_string(" cycles\n")
```

```

print_string("AVX (8-wide):      ")
print_dec(cycles_avx)
print_string(" cycles\n")

print_string("AVX-512 (16-wide): ")
print_dec(cycles_avx512)
print_string(" cycles\n")

print_string("AVX-512 Tiled:      ")
print_dec(cycles_tiled)
print_string(" cycles\n\n")

let speedup_avx = (cycles_sse as f64) / (cycles_avx as f64)
let speedup_avx512 = (cycles_sse as f64) / (cycles_avx512 as f64)
let speedup_tiled = (cycles_sse as f64) / (cycles_tiled as f64)

print_string("Speedup (AVX vs SSE):      ")
print_f64(speedup_avx)
print_string("x\n")
print_string("Speedup (AVX-512 vs SSE): ")
print_f64(speedup_avx512)
print_string("x\n")
print_string("Speedup (Tiled vs SSE):      ")
print_f64(speedup_tiled)
print_string("x\n")

//
=====
// HELPERS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

```

```

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut temp = value
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

fn print_f64(value: f64):
    let int_part = value as u64
    print_dec(int_part)
    print_char('.')
    let frac = (value - (int_part as f64)) * 100.0
    let frac_abs = if frac < 0.0: -frac else: frac
    let frac_int = frac_abs as u64
    if frac_int < 10:
        print_char('0')
    print_dec(frac_int)

fn main() -> u32:
    benchmark()
    return 0

```

Expected Output:

text

```

Matrix Multiplication SIMD Benchmark
=====

Initializing matrices...
Matrix size: 512x512

SSE (4-wide):      4250000000 cycles
AVX (8-wide):      2150000000 cycles
AVX-512 (16-wide): 1150000000 cycles
AVX-512 Tiled:     850000000 cycles

Speedup (AVX vs SSE):      1.98x
Speedup (AVX-512 vs SSE):  3.70x
Speedup (Tiled vs SSE):    5.00x

```

This concludes **Chapter 16: SIMD Vector Operations**. The chapter covered vector types for SSE, AVX, and AVX-512, construction and initialization methods, arithmetic operations (including FMA), horizontal operations (hadd, hmax, dot product), comparison operations and masks, permute/shuffle operations, load/store patterns (aligned, unaligned, gather, scatter, compress, expand), mathematical functions (sqrt, rcp, rsqrt), and a complete matrix multiplication benchmark demonstrating performance improvements across SIMD generations. SIMD is essential for high-performance computing, and lowl's integrated vector types make it both accessible and efficient.

Chapter 17: Memory Management and Protection

17.1 Introduction to Memory Management in Systems Programming

Memory management is the foundation of every operating system and systems program. Unlike user-space applications that rely on the operating system's memory allocator, lowl programs often run without any underlying OS (in kernels, bootloaders, or embedded systems). Therefore, lowl provides a complete, self-contained memory management system that operates directly on physical memory. This system includes a hierarchical physical memory allocator using a red-black tree of free regions, page table management for virtual memory, a protection violation visitor pattern for handling page faults, and support for advanced features like copy-on-write, memory-mapped files, and demand paging. The design prioritizes determinism, safety, and performance, with all operations having predictable worst-case execution times suitable for real-time systems.

17.2 Physical Memory Regions and Types

Physical memory on an x86_64 system is not uniform. The BIOS or UEFI firmware provides a memory map that categorizes regions by their purpose and availability. Some regions are usable RAM, some are reserved for hardware (MMIO, ACPI tables), some are ACPI reclaimable (can be reclaimed by the OS after boot), and some are defective (BADRAM). lowl's `MemoryType` enumeration captures these categories, and the `PhysicalAllocator` maintains a red-black tree of regions that can be split, merged, and queried. The allocator starts with the memory map from the bootloader (via Multiboot or E820) and builds an efficient data structure for allocation and deallocation.

lowl

```
// Memory region types from firmware
enum MemoryType:
    USABLE = 1           // Normal RAM, available for OS
    RESERVED = 2        // Reserved for hardware, BIOS, or firmware
    ACPI_RECLAIM = 3     // ACPI data that can be reclaimed after use
    ACPI_NVS = 4         // ACPI non-volatile storage (must be preserved)
    BAD_RAM = 5          // Defective memory, must not be used
    KERNEL = 6           // Memory containing kernel code/data
    KERNEL_MODULE = 7    // Loadable kernel module region
    USER = 8            // User-space memory (marked as user-accessible)
    SHARED = 9           // Shared memory between processes
    MMIO = 10            // Memory-mapped I/O (uncacheable)

// Page protection flags (from x86_64 page table entries)
enum PageFlags:
```

```

PRESENT = 1 << 0      // Page is present in physical memory
WRITABLE = 1 << 1     // Page can be written (read-only if 0)
USER = 1 << 2         // User-mode accessible (supervisor-only if 0)
WRITE_THROUGH = 1 << 3 // Write-through caching (vs write-back)
CACHE_DISABLE = 1 << 4 // Disable caching (for MMIO)
ACCESSED = 1 << 5     // Page has been accessed (set by CPU)
DIRTY = 1 << 6        // Page has been written (set by CPU)
HUGE = 1 << 7         // 2MB (PDPT) or 1GB (PML4) huge page
GLOBAL = 1 << 8       // Global page (not flushed from TLB on CR3
write)
NO_EXECUTE = 1 << 63  // Disable code execution on this page

// Memory region node for red-black tree
class MemoryRegionNode:
    base: u64          // Starting physical address
    length: u64         // Size in bytes
    mem_type: MemoryType // Region type
    left: ptr<MemoryRegionNode> = null
    right: ptr<MemoryRegionNode> = null
    parent: ptr<MemoryRegionNode> = null
    color: bool = true  // true=red, false=black (red-black tree)
    protection_flags: u64 // PageFlags for this region

impl MemoryRegionNode:
    fn new(base: u64, length: u64, mem_type: MemoryType) ->
MemoryRegionNode:
    return MemoryRegionNode{base, length, mem_type}

    fn split_at(address: u64) -> Option<ptr<MemoryRegionNode>>:
        if address <= this.base or address >= this.base + this.length:
            return Option.none()
        let left_len = address - this.base
        let right_len = this.base + this.length - address
        let left = MemoryRegionNode.new(this.base, left_len, this.mem_type)
        let right = MemoryRegionNode.new(address, right_len, this.mem_type)
        return Option.some(right) // Return the right part, left is this
node

```

17.3 Physical Allocator Implementation

The `PhysicalAllocator` class manages physical memory using a red-black tree of free regions. Allocation uses a best-fit algorithm: it finds the smallest region that can satisfy the request, splits it if necessary, and returns the allocated block. Deallocation reinserts the block back into the tree and merges adjacent free regions. The allocator also maintains a separate bitmap for tracking used pages (optional, for faster allocation of small objects). The red-black tree ensures $O(\log n)$ operations even with thousands of regions, making it suitable for long-running systems.

lowl

```

class PhysicalAllocator:
    private:
        root: ptr<MemoryRegionNode>
        total_ram: u64
        used_ram: u64
        page_size: u64 = 4096
        violation_handler: ptr<MemoryViolationVisitor> = null

    public:
        // Initialize allocator with memory map from bootloader
        fn init(memory_map: &array<MemoryMapEntry>) -> bool:
            for entry in memory_map:
                if entry.mem_type == MemoryType.USABLE:
                    this.register_region(entry.base, entry.length,
entry.mem_type)
            return true

        fn register_region(base: u64, length: u64, mem_type: MemoryType):
            let node = MemoryRegionNode.new(base, length, mem_type)
            this.root = this.rb_insert(this.root, node)
            this.total_ram = this.total_ram + length

        // Allocate physical pages
        fn alloc_pages(count: u64, alignment: u64, flags: PageFlags) ->
Option<u64>:
            let size = count * this.page_size
            let node = this.find_best_fit(this.root, size, alignment)
            if node.is_none():
                return Option.none()

            let mut n = node.unwrap()
            let address = n.base

            // Align address if needed
            let aligned = (address + alignment - 1) & ~(alignment - 1)
            let padding = aligned - address
            if padding > 0:
                // Split off the unaligned part
                let new_node = n.split_at(aligned)
                this.root = this.rb_insert(this.root,
MemoryRegionNode.new(n.base, padding, n.mem_type))
                n = new_node.unwrap()

            // Split the allocated portion
            if n.length > size:
                let remaining = MemoryRegionNode.new(n.base + size, n.length
- size, n.mem_type)
                this.root = this.rb_insert(this.root, remaining)

```

```

        n.length = size

        // Remove the allocated node from the tree
        this.root = this.rb_remove(this.root, n)
        this.used_ram = this.used_ram + size

        return Option.some(aligned)

    // Free previously allocated pages
    fn free_pages(address: u64, count: u64) -> bool:
        let size = count * this.page_size
        let node = MemoryRegionNode.new(address, size,
MemoryType.USABLE)
        this.root = this.rb_insert(this.root, node)
        this.merge_adjacent(node)
        this.used_ram = this.used_ram - size
        return true

    // Query protection for an address
    fn query_protection(address: u64) -> Option<PageFlags>:
        let node = this.find_region(address)
        if node.is_some():
            return Option.some(node.unwrap().protection_flags)
        return Option.none()

    // Set protection for a region
    fn protect_region(base: u64, length: u64, flags: PageFlags) -> bool:
        let node = this.find_region(base)
        if node.is_some():
            node.unwrap().protection_flags =
node.unwrap().protection_flags | flags
            this.update_page_table_permissions(base, length, flags)
            return true
        return false

    // Set violation handler
    fn set_violation_handler(handler: ptr<MemoryViolationVisitor>):
        this.violation_handler = handler

    // Handle page fault (called from page fault ISR)
    fn handle_page_fault(address: u64, error_code: u64) -> bool:
        if this.violation_handler != null:
            let is_write = (error_code & 2) != 0
            let is_user = (error_code & 4) != 0
            let is_exec = (error_code & 16) != 0

            if is_exec:
                return
            this.violation_handler.visit_exec_violation(address)

```

```

        elif is_write:
            return
    this.violation_handler.visit_write_violation(address)
        elif is_user:
            return
    this.violation_handler.visit_user_violation(address)
        else:
            return
    this.violation_handler.visit_read_violation(address)
    return false

private:
    // Red-black tree operations (insert, remove, find, rotations)
    fn rb_insert(root: ptr<MemoryRegionNode>, node:
ptr<MemoryRegionNode>) -> ptr<MemoryRegionNode>:
        // Standard red-black tree insertion
        root = this.bst_insert(root, node)
        this.rb_insert_fixup(root, node)
        return root

    fn find_best_fit(node: ptr<MemoryRegionNode>, size: u64, alignment:
u64) -> Option<ptr<MemoryRegionNode>>:
        if node == null:
            return Option.none()

        let aligned_base = (node.base + alignment - 1) & ~(alignment -
1)

        let usable_size = node.length - (aligned_base - node.base)

        let best_left = this.find_best_fit(node.left, size, alignment)
        let best_right = this.find_best_fit(node.right, size, alignment)

        let best_child = if best_left.is_some() and
best_right.is_some():
            let l = best_left.unwrap()
            let r = best_right.unwrap()
            if l.length < r.length: best_left else: best_right
        elif best_left.is_some():
            best_left
        elif best_right.is_some():
            best_right
        else:
            Option.none()

        if usable_size >= size:
            if best_child.is_some():
                let bc = best_child.unwrap()
                if bc.length < usable_size:
                    return best_child

```

```

        return Option.some(node)

    return best_child

fn merge_adjacent(node: ptr<MemoryRegionNode>):
    // Merge with previous node if contiguous
    let prev = this.predecessor(node)
    if prev != null and prev.base + prev.length == node.base:
        prev.length = prev.length + node.length
        this.root = this.rb_remove(this.root, node)
        node = prev

    // Merge with next node if contiguous
    let next = this.successor(node)
    if next != null and node.base + node.length == next.base:
        node.length = node.length + next.length
        this.root = this.rb_remove(this.root, next)

```

17.4 Memory Violation Visitor Pattern

The visitor pattern for memory violations allows different subsystems to register custom handlers for page faults. This is essential for implementing copy-on-write, lazy allocation, memory-mapped files, and garbage collection. The `MemoryViolationVisitor` base class defines virtual methods for each violation type, and subsystems override the methods they care about. The page fault handler calls the appropriate visitor method, which can decide to resolve the fault (by mapping a page, copying data, etc.) or indicate that the fault is fatal.

lowl

```

// Base visitor class for memory violations
abstract class MemoryViolationVisitor:
    public:
        // Called on read access to a page with no read permission
        virtual fn visit_read_violation(address: u64) -> bool:
            return false // Unhandled by default

        // Called on write access to a page with no write permission
        virtual fn visit_write_violation(address: u64) -> bool:
            return false

        // Called on execute access to a page with no execute permission
        virtual fn visit_exec_violation(address: u64) -> bool:
            return false

        // Called on user-mode access to a supervisor page
        virtual fn visit_user_violation(address: u64) -> bool:
            return false

// Copy-on-Write violation handler

```

```

class COWVisitor extends MemoryViolationVisitor:
  private:
    allocator: ptr<PhysicalAllocator>
    page_table: ptr<PageTable>

  public:
    fn new(alloc: ptr<PhysicalAllocator>, pt: ptr<PageTable>) ->
COWVisitor:
    this.allocator = alloc
    this.page_table = pt
    return this

  override fn visit_write_violation(address: u64) -> bool:
    let page_addr = address & ~4095

    // Get the current page table entry
    let pte = this.page_table.get_entry(page_addr)

    if (pte & PAGE_COW) != 0:
      // This is a copy-on-write page
      let new_page = this.allocator.alloc_pages(1, 4096,
PAGE_WRITABLE)
      if new_page.is_none():
        return false

      // Copy the original page content
      copy_memory(new_page.unwrap(), page_addr, 4096)

      // Update page table entry with new physical address and
writable flag
      this.page_table.set_entry(page_addr, new_page.unwrap(),
                                PAGE_PRESENT | PAGE_WRITABLE |
PAGE_USER)

      // Flush TLB
      invlpg(page_addr)
      return true

    return false

// Demand paging violation handler (zero-fill on first access)
class DemandPagingVisitor extends MemoryViolationVisitor:
  private:
    allocator: ptr<PhysicalAllocator>
    page_table: ptr<PageTable>

  public:
    fn new(alloc: ptr<PhysicalAllocator>, pt: ptr<PageTable>) ->
DemandPagingVisitor:

```

```

        this.allocator = alloc
        this.page_table = pt
        return this

    override fn visit_read_violation(address: u64) -> bool:
        let page_addr = address & ~4095

        // Allocate zero-filled page
        let new_page = this.allocator.alloc_pages(1, 4096, PAGE_PRESENT
| PAGE_WRITABLE | PAGE_USER)
        if new_page.is_none():
            return false

        // Zero the page
        zero_memory(new_page.unwrap(), 4096)

        // Update page table
        this.page_table.set_entry(page_addr, new_page.unwrap(),
                                PAGE_PRESENT | PAGE_WRITABLE |
PAGE_USER)

        invlpg(page_addr)
        return true

    override fn visit_write_violation(address: u64) -> bool:
        return this.visit_read_violation(address) // Same handling

// Memory-mapped file handler
class MemoryMappedFileVisitor extends MemoryViolationVisitor:
    private:
        file: ptr<File>
        offset: u64
        allocator: ptr<PhysicalAllocator>
        page_table: ptr<PageTable>

    public:
        fn new(f: ptr<File>, off: u64, alloc: ptr<PhysicalAllocator>, pt:
ptr<PageTable>) -> MemoryMappedFileVisitor:
            this.file = f
            this.offset = off
            this.allocator = alloc
            this.page_table = pt
            return this

    override fn visit_read_violation(address: u64) -> bool:
        let page_addr = address & ~4095
        let file_offset = this.offset + (page_addr - this.mapped_base)

        let new_page = this.allocator.alloc_pages(1, 4096, PAGE_PRESENT

```

```

    | PAGE_WRITABLE)
        if new_page.is_none():
            return false

        // Read page from file
        if not this.file.read_at(file_offset, new_page.unwrap(), 4096):
            this.allocator.free_pages(new_page.unwrap(), 1)
            return false

        this.page_table.set_entry(page_addr, new_page.unwrap(),
                                  PAGE_PRESENT | PAGE_USER |
(PAGE_WRITABLE if this.writable else 0))
        invlpg(page_addr)
        return true

```

17.5 Page Table Management

Virtual memory on x86_64 uses a 4-level page table hierarchy: PML4 (Page Map Level 4), PDPT (Page Directory Pointer Table), PD (Page Directory), and PT (Page Table). Each table has 512 entries of 8 bytes. The `PageTable` class provides an abstraction over this hardware structure, allowing mapping and unmapping of virtual addresses to physical pages, querying permissions, and handling page faults. The class uses the `invlpg` instruction to invalidate TLB entries when mappings change.

lowl

```

class PageTable:
    private:
        pml4: ptr<u64>          // Physical address of PML4
        allocator: ptr<PhysicalAllocator>

    public:
        fn new(alloc: ptr<PhysicalAllocator>) -> PageTable:
            // Allocate and zero PML4 page
            let pml4_page = alloc.alloc_pages(1, 4096, PAGE_PRESENT |
PAGE_WRITABLE)
            if pml4_page.is_none():
                panic("Failed to allocate PML4")
            this.pml4 = pml4_page.unwrap() as ptr<u64>
            zero_memory(this.pml4, 4096)
            this.allocator = alloc
            return this

        fn map(virt_addr: u64, phys_addr: u64, flags: u64) -> bool:
            let pml4_idx = (virt_addr >> 39) & 0x1FF
            let pdpt_idx = (virt_addr >> 30) & 0x1FF
            let pd_idx = (virt_addr >> 21) & 0x1FF
            let pt_idx = (virt_addr >> 12) & 0x1FF

```

```

// Get or create PDPT
let pdpt = this.get_or_create_table(this.pml4[pml4_idx], 1)
if pdpt == 0:
    return false

// Get or create PD
let pd = this.get_or_create_table(pdpt[pdpt_idx], 2)
if pd == 0:
    return false

// Get or create PT
let pt = this.get_or_create_table(pd[pd_idx], 3)
if pt == 0:
    return false

// Set PT entry
pt[pt_idx] = (phys_addr & ~0xFFF) | flags | PAGE_PRESENT
invlpg(virt_addr)
return true

fn unmap(virt_addr: u64) -> bool:
    let pml4_idx = (virt_addr >> 39) & 0x1FF
    let pdpt_idx = (virt_addr >> 30) & 0x1FF
    let pd_idx = (virt_addr >> 21) & 0x1FF
    let pt_idx = (virt_addr >> 12) & 0x1FF

    let pdpt = this.pml4[pml4_idx]
    if (pdpt & PAGE_PRESENT) == 0:
        return false

    let pd = (pdpt & ~0xFFF) as ptr<u64>
    if (pd[pdpt_idx] & PAGE_PRESENT) == 0:
        return false

    let pt = (pd[pdpt_idx] & ~0xFFF) as ptr<u64>
    if (pt[pd_idx] & PAGE_PRESENT) == 0:
        return false

    let pte_ptr = (pt[pd_idx] & ~0xFFF) as ptr<u64>
    if (pte_ptr[pt_idx] & PAGE_PRESENT) == 0:
        return false

    // Clear PTE and free page table if empty
    pte_ptr[pt_idx] = 0
    invlpg(virt_addr)
    return true

fn get_entry(virt_addr: u64) -> u64:
    let pml4_idx = (virt_addr >> 39) & 0x1FF

```

```

let pdpt_idx = (virt_addr >> 30) & 0x1FF
let pd_idx = (virt_addr >> 21) & 0x1FF
let pt_idx = (virt_addr >> 12) & 0x1FF

let pdpt = this.pml4[pml4_idx]
if (pdpt & PAGE_PRESENT) == 0:
    return 0

let pd = (pdpt & ~0xFFF) as ptr<u64>
if (pd[pdpt_idx] & PAGE_PRESENT) == 0:
    return 0

let pt = (pd[pdpt_idx] & ~0xFFF) as ptr<u64>
if (pt[pd_idx] & PAGE_PRESENT) == 0:
    return 0

let pte_ptr = (pt[pd_idx] & ~0xFFF) as ptr<u64>
return pte_ptr[pt_idx]

fn set_entry(virt_addr: u64, phys_addr: u64, flags: u64):
    this.map(virt_addr, phys_addr, flags)

fn delete():
    // Recursively free all page tables
    this.free_table(this.pml4, 0)

private:
    fn get_or_create_table(entry: u64, level: u64) -> ptr<u64>:
        if (entry & PAGE_PRESENT) != 0:
            return (entry & ~0xFFF) as ptr<u64>

        // Allocate new table
        let new_table = this allocator.alloc_pages(1, 4096, PAGE_PRESENT
| PAGE_WRITABLE)
        if new_table.is_none():
            return 0

        let table_ptr = new_table.unwrap() as ptr<u64>
        zero_memory(table_ptr, 4096)

        // Update the entry
        let entry_ptr = match level:
            case 1: &this.pml4[entry]
            case 2: (this.pml4[entry] & ~0xFFF) as ptr<u64>
            case 3: (this.pml4[entry] & ~0xFFF) as ptr<u64>
            default: null

        if entry_ptr != null:
            *entry_ptr = (new_table.unwrap() as u64) | PAGE_PRESENT |

```

PAGE_WRITABLE | PAGE_USER

```

    return table_ptr

fn free_table(table: ptr<u64>, level: u64):
    if table == null:
        return
    for i in 0..512:
        let entry = table[i]
        if (entry & PAGE_PRESENT) != 0 and level < 3:
            let child = (entry & ~0xFFF) as ptr<u64>
            this.free_table(child, level + 1)
    this allocator.free_pages(table as u64, 1)

```

17.6 Complete Chapter Example: Kernel Memory Manager

This example demonstrates a complete memory manager for a kernel, integrating the physical allocator, page table manager, and violation visitors.

lowl

```

// memmanager.lowl - Kernel Memory Manager
// Compile: lowlc memmanager.lowl -o memmanager.asm -O2

//
=====
// MEMORY MANAGER CLASS
//
=====

class KernelMemoryManager:
    private:
        phys_alloc: PhysicalAllocator
        kernel_page_table: PageTable
        cow_visitor: COWVisitor
        zero_visitor: DemandPagingVisitor
        current_visitor: ptr<MemoryViolationVisitor>

    public:
        fn new(memory_map: &array<MemoryMapEntry>) -> KernelMemoryManager:
            this.phys_alloc.init(memory_map)
            this.kernel_page_table = PageTable.new(&this.phys_alloc)
            this.cow_visitor = COWVisitor.new(&this.phys_alloc,
&this.kernel_page_table)
            this.zero_visitor = DemandPagingVisitor.new(&this.phys_alloc,
&this.kernel_page_table)
            this.current_visitor = &this.zero_visitor
            this.phys_alloc.set_violation_handler(&this.zero_visitor)
            return this

```

```

fn alloc_user_pages(virt_addr: u64, count: u64, writable: bool) ->
bool:
    let flags = PAGE_PRESENT | PAGE_USER
    if writable:
        flags = flags | PAGE_WRITABLE

    for i in 0..count:
        let phys = this.phys_alloc.alloc_pages(1, 4096, flags)
        if phys.is_none():
            return false
        this.kernel_page_table.map(virt_addr + i * 4096,
phys.unwrap(), flags)

    return true

fn free_user_pages(virt_addr: u64, count: u64):
    for i in 0..count:
        let pte = this.kernel_page_table.get_entry(virt_addr + i *
4096)

        if (pte & PAGE_PRESENT) != 0:
            let phys = pte & ~0xFFF
            this.phys_alloc.free_pages(phys, 1)
            this.kernel_page_table.unmap(virt_addr + i * 4096)

fn enable_copy_on_write():
    this.current_visitor = &this.cow_visitor
    this.phys_alloc.set_violation_handler(&this.cow_visitor)

fn enable_demand_paging():
    this.current_visitor = &this.zero_visitor
    this.phys_alloc.set_violation_handler(&this.zero_visitor)

fn get_stats() -> (u64, u64, u64):
    return (this.phys_alloc.total_ram, this.phys_alloc.used_ram,
        this.phys_alloc.total_ram - this.phys_alloc.used_ram)

//
=====
// PAGE FAULT HANDLER
//
=====

#[interrupt]
fn page_fault_handler():
    let fault_address = read_cr2()
    let error_code = asm("mov rax, [rsp+16]") as u64

    if not kernel_mem_manager.handle_page_fault(fault_address, error_code):

```

```

    print_string("\n!!! UNHANDLED PAGE FAULT !!!\n")
    print_string("Address: 0x")
    print_hex(fault_address)
    print_string("\nError code: 0x")
    print_hex(error_code)
    print_string("\n")
    while true:
        halt()

//
=====
// DEMONSTRATION
//
=====

static kernel_mem_manager: KernelMemoryManager

fn main() -> u32:
    print_string("=== Kernel Memory Manager Demo ===\n\n")

    // Create memory map for a typical system with 256MB
    let memory_map: array<MemoryMapEntry, 3> = [
        MemoryMapEntry{0x100000, 256 * 1024 * 1024, MemoryType.USABLE}, //
1MB to 257MB
        MemoryMapEntry{0, 0x9FC00, MemoryType.RESERVED},
// BIOS area
        MemoryMapEntry{0x100000000, 0x100000000, MemoryType.USABLE}
// 4GB+ region
    ]

    kernel_mem_manager = KernelMemoryManager.new(&memory_map)

    let (total, used, free) = kernel_mem_manager.get_stats()
    print_string("Physical memory:\n")
    print_string("  Total: ")
    print_dec(total / (1024 * 1024))
    print_string(" MB\n")
    print_string("  Used: ")
    print_dec(used / (1024 * 1024))
    print_string(" MB\n")
    print_string("  Free: ")
    print_dec(free / (1024 * 1024))
    print_string(" MB\n\n")

    // Allocate user pages
    print_string("Allocating 16 user pages at 0x400000...\n")
    if kernel_mem_manager.alloc_user_pages(0x400000, 16, true):
        print_string("  Allocation successful\n")

```

```

// Demonstrate demand paging
print_string("\nAccessing allocated pages (demand paging)...\n")
let user_ptr = 0x400000 as ptr_mut<u64>
for i in 0..16:
    user_ptr[i] = 0xDEADBEEF // This will trigger page faults
    if i % 4 == 0:
        print_string("  Page ")
        print_dec(i)
        print_string(" written\n")

// Free pages
print_string("\nFreeing user pages...\n")
kernel_mem_manager.free_user_pages(0x400000, 16)
print_string("  Free complete\n")

return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut temp = value
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    while temp > 0:
        digits[i] = (temp % 10) as u8

```

```

    temp = temp / 10
    i = i + 1
  for j in i..0 step -1:
    print_char('0' + digits[j - 1])

fn print_hex(value: u64):
  let hex_digits = "0123456789ABCDEF"
  for i in 60..0 step -4:
    let nibble = (value >> i) & 0xF
    if nibble != 0 or i == 0:
      print_char(hex_digits[nibble as u64] as u8)
  let last = value & 0xF
  print_char(hex_digits[last as u64] as u8)

```

Expected Output:

text

```

=== Kernel Memory Manager Demo ===

Physical memory:
  Total: 256 MB
  Used: 1 MB
  Free: 255 MB

Allocating 16 user pages at 0x400000...
  Allocation successful

Accessing allocated pages (demand paging)...
  Page 0 written
  Page 4 written
  Page 8 written
  Page 12 written

Freeing user pages...
  Free complete

```

This concludes **Chapter 17: Memory Management and Protection**. The chapter covered physical memory region types, the red-black tree allocator, page table management, the memory violation visitor pattern (for copy-on-write, demand paging, and memory-mapped files), and a complete kernel memory manager. Memory management is the foundation of any operating system, and lowl provides the tools to implement it efficiently and safely.

Chapter 18: Module System and Executable Loader

18.1 Introduction to Dynamic Modularity

Modern operating systems and large systems programs require dynamic extensibility: the ability to load code at runtime after the core system is already running. Device drivers, file systems, security modules, and application plugins all benefit from being loaded on demand rather than compiled into the base image. lowl's module system provides a complete framework for building, loading, and managing dynamic modules. Modules are compiled separately from the kernel or main program, using the `-f kernel` output format, and contain special entry points (`module_init` and `module_exit`) that the loader calls. The loader handles relocations, resolves imports between modules, manages protection rings (modules can run at different privilege levels), and provides a stable API for symbol resolution. This chapter covers the complete module system, from the module header format to runtime loading and cross-module communication.

18.2 Module Header and Format

Every lowl module begins with a `ModuleHeader` structure that contains metadata necessary for loading. The header includes a magic number (0x4C4F574C = "LOWL" in ASCII), version information, the entry point offsets for initialization and cleanup, section sizes for text, data, rodata, and bss, symbol tables for imports and exports, and a checksum for integrity verification. The module format is position-independent: all code uses relative addressing so the module can be loaded at any virtual address. The loader reads this header, allocates memory for each section (respecting alignment requirements), copies the section contents, performs relocations, resolves symbols, and finally calls the module's initialization function.

lowl

```
// Module header structure (64 bytes, at start of module file)
#[packed]
struct ModuleHeader:
    magic: u32 = 0x4C4F574C           // "LOWL" magic number
    version: u32 = 0x00020001         // Major=2, Minor=1, Patch=0
    entry_point: u64 = 0              // Offset to module_init (relative to
base)
    exit_point: u64 = 0               // Offset to module_exit (relative to
base)
    text_offset: u64 = 0              // Offset to .text section
    text_size: u64 = 0               // Size of .text section
    data_offset: u64 = 0             // Offset to .data section
    data_size: u64 = 0               // Size of .data section
    rodata_offset: u64 = 0           // Offset to .rodata section
    rodata_size: u64 = 0            // Size of .rodata section
    bss_size: u64 = 0               // Size of .bss section (zero-
initialized)
    symtab_offset: u64 = 0           // Offset to symbol table
    symtab_count: u64 = 0           // Number of symbols
    strtab_offset: u64 = 0          // Offset to string table
    strtab_size: u64 = 0            // Size of string table
```

```

import_count: u64 = 0           // Number of imported symbols
export_count: u64 = 0           // Number of exported symbols
protection_ring: u8 = 3         // Ring level (0=kernel, 3=user)
checksum: u32 = 0               // CRC32 of module (excluding header)
_reserved: array<u8, 19> = [0; 19] // Padding to 64 bytes

// Symbol table entry (32 bytes)
#[packed]
struct ModuleSymbol:
    name_offset: u32           // Offset into string table
    value: u64                 // Address (filled at load time)
    size: u64                  // Size of symbol (0 for functions)
    type: u8                   // 1=import, 2=export, 3=local
    binding: u8                 // 1=global, 2=weak, 3=local
    section: u16                // Section index
(1=text,2=data,3=rodata,4=bss)

```

18.3 Module Compilation and Linking

To produce a loadable module, the lowl compiler is invoked with the `-f kernel` flag. This generates position-independent code with a module header. The linker then combines the sections and produces the final `.ko` (kernel object) file. Modules can export symbols using the `#[export]` attribute, making them available to other modules and the kernel. Imports are resolved when the module is loaded: the loader looks up each imported symbol in the kernel's symbol table or in already-loaded modules.

lowl

```

// Example module: simple_driver.lowl
// Compile: lowlc simple_driver.lowl -o driver.asm -f kernel

// Export the initialization and cleanup functions
#[export]
fn module_init() -> u32:
    print_string("Simple driver loaded!\n")
    return 0 // Success

#[export]
fn module_exit() -> u32:
    print_string("Simple driver unloaded!\n")
    return 0

// Export a function for other modules to use
#[export]
fn driver_operation(value: u64) -> u64:
    return value * 2

// Internal function (not exported)
fn internal_helper():

```

```
// Only accessible within this module

// Global variable (exported)
#[export]
static driver_status: u32 = 1
```

18.4 Module Loader Implementation

The `ModuleLoader` class is responsible for loading modules into memory. It reads the module file, validates the header, allocates memory for each section (respecting alignment and page boundaries), copies the section data, zeroes the BSS, performs relocations, resolves imports against the kernel's symbol table and other loaded modules, and finally calls the module's initialization function. The loader maintains a list of loaded modules to resolve cross-module dependencies and to support unloading.

lowl

```
class ModuleLoader:
    private:
        loaded_modules: rb_map<string, ptr<LoadedModule>>
        kernel_symbols: rb_map<string, u64>
        allocator: ptr<PhysicalAllocator>
        page_table: ptr<PageTable>

    public:
        fn new(alloc: ptr<PhysicalAllocator>, pt: ptr<PageTable>) ->
ModuleLoader:
        this.allocator = alloc
        this.page_table = pt
        this.loaded_modules = rb_map<string,
ptr<LoadedModule>>.new(compare_string)
        this.kernel_symbols = this.build_kernel_syntab()
        return this

        // Load a module from a file
        fn load(module_path: string, ring: ProtectionRing) ->
Option<ptr<LoadedModule>>:
            // Open and read the module file
            let file = open_file(module_path, FILE_MODE_READ)
            if file == null:
                return Option.none()

            // Read the header
            let header: ModuleHeader
            if not file.read(&header, sizeof(ModuleHeader)):
                file.close()
                return Option.none()

            // Validate header
```

```

if header.magic != 0x4C4F574C:
    file.close()
    return Option.none()

if header.version != 0x00020001:
    print_string("Module version mismatch\n")
    file.close()
    return Option.none()

// Allocate memory for the module
let module = this.allocate_module(&header, ring)
if module == null:
    file.close()
    return Option.none()

// Load sections
if not this.load_sections(file, &header, module):
    this.unload(module)
    file.close()
    return Option.none()

// Perform relocations
if not this.relocate_module(module):
    this.unload(module)
    file.close()
    return Option.none()

// Resolve imports
if not this.resolve_imports(module):
    this.unload(module)
    file.close()
    return Option.none()

// Register exports
this.register_exports(module)

// Call module_init
let init_func = (module.base + header.entry_point) as fn() ->

let result = init_func()

if result != 0:
    this.unload(module)
    file.close()
    return Option.none()

// Add to loaded modules list
this.loaded_modules.insert(module_path, module)

```

u32

```

        file.close()
        return Option.some(module)

// Unload a module
fn unload(module: ptr<LoadedModule>):
    if module == null:
        return

    // Call module_exit
    if module.exit_point != 0:
        let exit_func = (module.base + module.exit_point) as fn() ->

u32        exit_func()

    // Unregister exports
    this.unregister_exports(module)

    // Free memory
    if module.text != 0:
        this.allocator.free_pages(module.text, module.text_pages)
    if module.data != 0:
        this.allocator.free_pages(module.data, module.data_pages)
    if module.rodata != 0:
        this.allocator.free_pages(module.rodata,
module.rodata_pages)

    // Remove from list
    this.loaded_modules.remove(module.name)

    physical_free(module)

// Resolve a symbol from any loaded module
fn resolve_symbol(name: string) -> Option<u64>:
    // Check kernel symbols first
    let opt = this.kernel_symbols.find(name)
    if opt.is_some():
        return opt

    // Check loaded modules
    for module in this.loaded_modules.values():
        let sym = module.exports.find(name)
        if sym.is_some():
            return sym

    return Option.none()

private:
    fn allocate_module(header: &ModuleHeader, ring: ProtectionRing) ->
ptr<LoadedModule>:

```

```

        let module = physical_alloc(sizeof(LoadedModule), 8) as
ptr<LoadedModule>
        if module == null:
            return null

        // Calculate pages needed for each section
        let text_pages = (header.text_size + 4095) / 4096
        let data_pages = (header.data_size + 4095) / 4096
        let rodata_pages = (header.rodata_size + 4095) / 4096
        let bss_pages = (header.bss_size + 4095) / 4096

        // Set protection flags based on ring level
        let text_flags = PAGE_PRESENT | (if ring ==
ProtectionRing.RING0_KERNEL: PAGE_WRITABLE else: PAGE_USER)
        let data_flags = PAGE_PRESENT | PAGE_WRITABLE | (if ring !=
ProtectionRing.RING0_KERNEL: PAGE_USER else: 0)
        let rodata_flags = PAGE_PRESENT | (if ring !=
ProtectionRing.RING0_KERNEL: PAGE_USER else: 0)

        // Allocate pages
        module.text = this.allocator.alloc_pages(text_pages, 4096,
text_flags)
        module.data = this.allocator.alloc_pages(data_pages, 4096,
data_flags)
        module.rodata = this.allocator.alloc_pages(rodata_pages, 4096,
rodata_flags)
        let bss_addr = this.allocator.alloc_pages(bss_pages, 4096,
data_flags)

        if module.text == 0 or module.data == 0 or module.rodata == 0 or
bss_addr == 0:
            // Allocation failed: clean up
            if module.text != 0: this.allocator.free_pages(module.text,
text_pages)
            if module.data != 0: this.allocator.free_pages(module.data,
data_pages)
            if module.rodata != 0:
this.allocator.free_pages(module.rodata, rodata_pages)
            if bss_addr != 0: this.allocator.free_pages(bss_addr,
bss_pages)

            physical_free(module)
            return null

        // Map pages into virtual address space
        // (Simplified - actual implementation would create a proper
mapping)
        module.base = module.text
        module.text_size = header.text_size
        module.data_size = header.data_size

```

```

    module.rodata_size = header.rodata_size
    module.bss_size = header.bss_size
    module.bss = bss_addr
    module.exit_point = header.exit_point
    module.name = "" // To be filled

    // Zero BSS
    zero_memory(module.bss, header.bss_size)

    return module

fn load_sections(file: ptr<File>, header: &ModuleHeader, module:
ptr<LoadedModule>) -> bool:
    // Seek to and read .text section
    file.seek(header.text_offset)
    if not file.read(module.text, header.text_size):
        return false

    // Read .data section
    file.seek(header.data_offset)
    if not file.read(module.data, header.data_size):
        return false

    // Read .rodata section
    file.seek(header.rodata_offset)
    if not file.read(module.rodata, header.rodata_size):
        return false

    // Read symbol table and string table (for debugging)
    if header.symtab_offset != 0:
        module.symtab = physical_alloc(
            header.symtab_count * sizeof(ModuleSymbol),
            8
        ) as ptr<ModuleSymbol>
        file.seek(header.symtab_offset)
        file.read(module.symtab, header.symtab_count *
sizeof(ModuleSymbol))

    if header.strtab_offset != 0:
        module.strtab = physical_alloc(header.strtab_size, 8) as
ptr<u8>

        file.seek(header.strtab_offset)
        file.read(module.strtab, header.strtab_size)

    return true

```

18.5 Executable Loader and Protection Rings

The `ExecutableLoader` extends the module loader to support loading full executables (not just kernel modules) and running them at different protection rings. This enables user-space processes to be loaded and managed by the kernel. The loader sets up page tables for the process (with user-space mappings), creates a process control block, sets up the initial stack and arguments, and transfers control to the entry point. The `syscall` instruction is used for transitions between ring 3 (user) and ring 0 (kernel).

lowl

```
class ExecutableLoader:
    private:
        module_loader: ModuleLoader
        processes: rb_map<u64, ptr<Process>>
        next_pid: u64 = 1

    public:
        fn new(alloc: ptr<PhysicalAllocator>, pt: ptr<PageTable>) ->
ExecutableLoader:
        this.module_loader = ModuleLoader.new(alloc, pt)
        this.processes = rb_map<u64, ptr<Process>>.new(compare_u64)
        return this

        fn load_executable(path: string, argv: &array<string>) ->
Option<u64>:
        // Load the executable as a module (runs at ring 3)
        let module = this.module_loader.load(path,
ProtectionRing.RING3_USER)
        if module.is_none():
            return Option.none()

        let m = module.unwrap()

        // Create process structure
        let proc = physical_alloc(sizeof(Process), 8) as ptr<Process>
        proc.pid = this.next_pid
        this.next_pid = this.next_pid + 1
        proc.module = m
        proc.state = ProcessState.READY

        // Set up user-space stack
        let stack_pages = 16 // 64KB stack
        proc.stack_base = this.module_loader.allocator.alloc_pages(
            stack_pages, 4096, PAGE_PRESENT | PAGE_WRITABLE | PAGE_USER
        )
        proc.stack_top = proc.stack_base + stack_pages * 4096

        // Set up argument vectors on stack
        let sp = proc.stack_top
```

```

sp = sp - 8 // Null terminator for argv
for i in argv.len()..0 step -1:
    let arg_ptr = proc.stack_base + (i * 256) // Simplified
    copy_memory(arg_ptr, argv[i].c_str(), argv[i].len() + 1)
    sp = sp - 8
    (sp as ptr<u64>)[0] = arg_ptr
sp = sp - 8
(sp as ptr<u64>)[0] = argv.len() as u64 // argc

proc.user_rsp = sp
proc.user_rip = m.base + 0 // Entry point at base address

// Create user page table (copy of kernel page table with user
mappings)
proc.page_table = this.create_user_page_table()

// Store process
this.processes.insert(proc.pid, proc)

return Option.some(proc.pid)

fn run_process(pid: u64) -> bool:
    let opt = this.processes.find(pid)
    if opt.is_none():
        return false

    let proc = opt.unwrap()
    proc.state = ProcessState.RUNNING

    // Switch to process page table
    write_cr3(proc.page_table.pm14 as u64)

    // Switch to user mode using sysret or iret
    let user_rip = proc.user_rip
    let user_rsp = proc.user_rsp

    asm("
        mov ax, 0x23 // User data segment selector (ring 3)
        mov ds, ax
        mov es, ax
        mov fs, ax
        mov gs, ax

        push 0x23 // SS (user stack segment)
        push %1 // RSP (user stack pointer)
        pushfq // RFLAGS
        push 0x1B // CS (user code segment, ring 3)
        push %0 // RIP (user entry point)
        iretq

```

```

        " : : "r"(user_rip), "r"(user_rsp))

    return true

fn syscall_handler(syscall_num: u64, arg1: u64, arg2: u64, arg3:
u64) -> u64:
    // Called from user space via SYSCALL instruction
    switch (syscall_num):
        case 0: // write
            return syscall_write(arg1 as i32, arg2 as ptr<u8>, arg3
as u64)

        case 1: // read
            return syscall_read(arg1 as i32, arg2 as ptr<u8>, arg3
as u64)

        case 2: // open
            return syscall_open(arg1 as string, arg2 as u32)
        case 3: // close
            return syscall_close(arg1 as i32)
        case 4: // exit
            syscall_exit(arg1 as i32)
            return 0
        default:
            return -1

// Process control block
struct Process:
    pid: u64
    module: ptr<LoadedModule>
    page_table: ptr<PageTable>
    state: ProcessState
    stack_base: u64
    stack_top: u64
    user_rip: u64
    user_rsp: u64

```

18.6 System Call Interface

The system call interface is the primary mechanism for user-space programs to request kernel services. lowl provides a fast syscall path using the `syscall` and `sysret` instructions. The kernel sets up the `STAR`, `LSTAR`, and `SFMASK` MSRs to configure the syscall entry point and the flags that are cleared when entering the kernel. The `#[syscall]` attribute marks functions that can be called from user space.

lowl

```

// System call handler in the kernel
#[syscall]
fn syscall_dispatcher(num: u64, arg1: u64, arg2: u64, arg3: u64, arg4: u64,
arg5: u64) -> u64:

```

```

switch (num):
    case SYSCALL_READ:
        return syscall_read(arg1 as i32, arg2 as ptr<u8>, arg3 as u64)
    case SYSCALL_WRITE:
        return syscall_write(arg1 as i32, arg2 as ptr<u8>, arg3 as u64)
    case SYSCALL_OPEN:
        return syscall_open(arg1 as string, arg2 as u32)
    case SYSCALL_CLOSE:
        return syscall_close(arg1 as i32)
    case SYSCALL_MMAP:
        return syscall_mmap(arg1 as ptr, arg2 as u64, arg3 as u32, arg4
as u32, arg5 as i32, arg6 as u64)
    case SYSCALL_EXIT:
        syscall_exit(arg1 as i32)
        return 0
    default:
        return -1

// Setup syscall MSRs (called during kernel initialization)
fn setup_syscall():
    // STAR: Set segment selectors for syscall/sysret
    // Bits 63-48: Kernel CS base (0x08)
    // Bits 47-32: Kernel SS base (0x10)
    // Bits 31-16: User CS base (0x1B)
    // Bits 15-0:  User SS base (0x23)
    let star = (0x08 << 48) | (0x10 << 32) | (0x1B << 16) | 0x23
    write_msr(0xC0000081, star)

    // LSTAR: Syscall entry point
    write_msr(0xC0000082, &syscall_dispatcher as u64)

    // SFMASK: Clear IF (interrupts) and DF (direction flag) on syscall
    write_msr(0xC0000084, 0x200 | 0x400)

```

18.7 Complete Chapter Example: Simple Module and Loader

This example demonstrates a complete module system with a loader that can load and run modules, resolve imports, and unload modules.

lowl

```

// loader_demo.lowl - Complete Module System Demo
// Compile: lowlc loader_demo.lowl -o loader.asm -O2

//
=====
// MODULE INTERFACE
//
=====

```

```
// Module to be loaded (saved as "math_module.ko")
// Compile separately: lowlc math_module.lowl -f kernel

#[export]
fn module_init() -> u32:
    print_string("Math module loaded\n")
    return 0

#[export]
fn module_exit() -> u32:
    print_string("Math module unloaded\n")
    return 0

#[export]
fn add(a: i64, b: i64) -> i64:
    return a + b

#[export]
fn mul(a: i64, b: i64) -> i64:
    return a * b

static call_count: u64 = 0

#[export]
fn get_call_count() -> u64:
    return call_count

//
=====
// LOADER DEMONSTRATION
//
=====

fn main() -> u32:
    print_string("=== Module Loader Demo ===\n\n")

    // Initialize memory allocator
    let allocator = PhysicalAllocator.new()
    let memory_map = get_memory_map_from_boot()
    allocator.init(&memory_map)

    // Initialize page table
    let page_table = PageTable.new(&allocator)

    // Create module loader
    let loader = ModuleLoader.new(&allocator, &page_table)
```

```

// Load math module
print_string("Loading math_module.ko...\n")
let opt = loader.load("math_module.ko", ProtectionRing.RING0_KERNEL)

if opt.is_none():
    print_string("Failed to load module\n")
    return 1

let module = opt.unwrap()

// Resolve and call exported functions
print_string("\nResolving symbols...\n")

let add_fn = loader.resolve_symbol("add")
let mul_fn = loader.resolve_symbol("mul")
let get_count_fn = loader.resolve_symbol("get_call_count")

if add_fn.is_some() and mul_fn.is_some():
    let add = add_fn.unwrap() as fn(i64, i64) -> i64
    let mul = mul_fn.unwrap() as fn(i64, i64) -> i64

    print_string("\nCalling module functions:\n")
    print_string("  add(10, 20) = ")
    print_dec(add(10, 20))
    print_string("\n")
    print_string("  mul(5, 6) = ")
    print_dec(mul(5, 6))
    print_string("\n")

// Unload the module
print_string("\nUnloading module...\n")
loader.unload(module)

print_string("\nDemo complete!\n")

return 0

//
=====

// HELPER FUNCTIONS
//
=====

struct MemoryMapEntry:
    base: u64
    length: u64
    mem_type: u32

fn get_memory_map_from_boot() -> array<MemoryMapEntry, 3>:

```

```

// Simplified - in real system, this would come from Multiboot or E820
return [
    MemoryMapEntry{0x100000, 256 * 1024 * 1024, 1},
    MemoryMapEntry{0, 0x9FC00, 2},
    MemoryMapEntry{0x10000000, 0x10000000, 1}
]

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: i64):
    if value == 0:
        print_char('0')
        return
    let mut temp = value
    if temp < 0:
        print_char('-')
        temp = -temp
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])

fn compare_string(a: string, b: string) -> i8:
    let min_len = min(a.len(), b.len())
    for i in 0..min_len:
        if a[i] < b[i]: return -1
        elif a[i] > b[i]: return 1
    if a.len() < b.len(): return -1
    elif a.len() > b.len(): return 1
    return 0

```

```
fn compare_u64(a: u64, b: u64) -> i8:
    if a < b: return -1
    elif a > b: return 1
    return 0
```

Expected Output:

text

```
=== Module Loader Demo ===

Loading math_module.ko...
Math module loaded

Resolving symbols...

Calling module functions:
    add(10, 20) = 30
    mul(5, 6) = 30

Unloading module...
Math module unloaded

Demo complete!
```

This concludes **Chapter 18: Module System and Executable Loader**. The chapter covered the module header format, compilation and linking of modules, the module loader implementation (section loading, relocations, import resolution, export registration), the executable loader for user-space processes, protection ring transitions, the system call interface, and a complete demonstration of loading and using a module. Dynamic extensibility is a cornerstone of modern systems, and lowl's module system provides it with safety and performance.

Chapter 19: Optimizer and Pragmas

19.1 Introduction to Optimization in lowl

Optimization is the process of transforming source code into equivalent but more efficient machine code. lowl provides four optimization levels (O0 through O3) that balance compilation time, code size, and execution speed. Unlike many compilers that apply optimizations indiscriminately, lowl's optimizer is designed with systems programming in mind: optimizations must never change the observable behavior of programs that interact with hardware. This means that volatile accesses (via `mmio_ptr`), memory-mapped I/O operations, and inline assembly are preserved exactly as written. The optimizer operates on the abstract syntax tree (AST), performing transformations that are sound for all lowl programs. This chapter covers each optimization level in detail, the specific transformations applied, and the pragmas that give programmers fine-grained control over optimization.

19.2 Optimization Level O0: No Optimization

O0 is the default when debugging is the primary concern. At O0, the compiler generates code that directly corresponds to the source structure: each variable has a stack slot, each statement is compiled independently, and no operations are reordered or eliminated. This makes debugging straightforward because variables can be inspected at breakpoints and execution flow matches source line order. O0 is also useful when you need to measure the "naive" performance of an algorithm before applying optimizations. The generated code is larger and slower than optimized code, but it is predictable and easy to understand.

lowI

```
// Example code compiled at O0
fn slow_multiply(a: u64, b: u64) -> u64:
    let result = 0
    for i in 0..b:
        result = result + a
    return result

// O0 assembly characteristics:
// - Each variable has a dedicated stack slot (rbp-8, rbp-16, etc.)
// - No instruction reordering
// - No function inlining
// - Loop is not unrolled
// - Simple addressing modes only
```

19.3 Optimization Level O1: Basic Optimizations

O1 performs local optimizations that do not change the overall structure of the code. Constant folding replaces expressions like `2 + 3` with `5` at compile time. Dead code elimination removes statements that have no effect, such as assignments to variables that are never read. Redundant load elimination avoids loading the same value multiple times from memory. Simple constant propagation replaces variable references with their known constant values. These transformations reduce code size and improve execution speed without requiring extensive analysis, making O1 suitable for most production builds where compile time matters.

lowI

```
// O1 optimization examples

// Constant folding - computed at compile time
let pi_approx = 3.14159 * 2.0 // Optimized to 6.28318

// Dead code elimination - unreachable code removed
if false:
    do_something() // Removed entirely

// Constant propagation - variable replaced with constant
let x = 42
let y = x + 1 // Optimized to y = 43
```

```
// Redundant load elimination
let a = array[10]
let b = array[10] // Second load eliminated, a reused
```

19.4 Optimization Level O2: Aggressive Optimizations

O2 includes all O1 optimizations plus more aggressive transformations. Function inlining expands small functions at call sites, eliminating call overhead and enabling further optimizations. Loop invariant code motion moves computations that do not change during loop execution outside the loop. Strength reduction replaces expensive operations with cheaper ones (e.g., multiplication by constant replaced by shifts and adds). Basic SIMD vectorization automatically converts scalar loops to SIMD operations when the compiler can prove it is safe. O2 is the default optimization level for production builds, offering excellent performance without excessive compile time.

lowl

```
// O2 optimization examples

// Function inlining
inline fn square(x: u64) -> u64:
    return x * x

let s = square(10) // Inlined to: let s = 10 * 10

// Loop invariant code motion
for i in 0..1000000:
    let c = expensive_computation() // Called once before loop
    arr[i] = arr[i] + c              // c is invariant

// Strength reduction
for i in 0..n:
    arr[i * 8] = 0 // Optimized to: arr[0] = 0; arr[8] = 0; arr[16] = 0;
    ...
    // using pointer arithmetic instead of multiplication

// Basic SIMD vectorization
for i in 0..1000:
    c[i] = a[i] + b[i] // Converted to vec8_f32 load/add/store
```

19.5 Optimization Level O3: Full Optimizations

O3 includes all O2 optimizations and adds transformations that may increase code size but improve performance further. Loop unrolling replicates loop bodies to reduce the number of branch instructions. Cross-block SIMD vectorization identifies vectorizable patterns across multiple loops. Block fusion merges adjacent loops that operate on the same data to improve cache locality. Aggressive inlining inlines functions that are larger and called more frequently. Software prefetching inserts prefetch instructions to hide memory latency. Speculative execution hints guide the processor's branch predictor. O3 is recommended for performance-critical code where code size is not a concern.

lowl

```
// 03 optimization examples

// Loop unrolling
for i in 0..100 step 4:
    // Unrolled: processes 4 iterations per loop
    arr[i] = arr[i] * 2
    arr[i+1] = arr[i+1] * 2
    arr[i+2] = arr[i+2] * 2
    arr[i+3] = arr[i+3] * 2

// Block fusion (fusion of adjacent loops)
// Original:
for i in 0..n: a[i] = b[i] + c[i]
for i in 0..n: d[i] = a[i] * e[i]
// Fused:
for i in 0..n: d[i] = (b[i] + c[i]) * e[i]

// Software prefetching
for i in 0..n:
    prefetch(&array[i + 64], 0) // Prefetch future elements
    process(array[i])
```

19.6 SIMD Optimization Pragas

The `#pragma simd` directive controls how the compiler vectorizes loops. It can override the default SIMD level (SSE, AVX, or AVX-512) and provide hints about vectorization. The `#pragma simd(assume_aligned)` tells the compiler that a pointer is aligned to the specified byte boundary, enabling aligned loads/stores. `#pragma simd(without_masking)` disables generation of masked instructions (AVX-512 only). `#pragma simd(linear)` specifies a variable that increases linearly within the loop. These pragmas are essential for getting maximum performance from numeric code.

lowl

```
// SIMD pragma examples

// Force AVX-512 vectorization for a specific loop
#pragma simd(AVX512)
fn vector_add(a: ptr<f32>, b: ptr<f32>, c: ptr<f32>, n: u64):
    for i in 0..n:
        c[i] = a[i] + b[i]

// Assume 64-byte alignment for AVX-512
#pragma simd(assume_aligned(64))
fn aligned_process(data: ptr<f32>, n: u64):
    for i in 0..n:
        data[i] = data[i] * 2.0
```

```
// Vectorize with reduction
#pragma simd(reduction(+:sum))
fn sum_array(data: ptr<f32>, n: u64) -> f32:
    let mut sum = 0.0
    for i in 0..n:
        sum = sum + data[i]
    return sum

// Vectorize with linear variable
#pragma simd(linear(i:4))
fn strided_access(data: ptr<f32>, n: u64):
    for i in 0..n:
        data[i * 4] = data[i * 4] * 2.0
```

19.7 Loop Optimization Pragas

Loop pragmas give the programmer control over how loops are optimized. `#pragma unroll` controls loop unrolling (with optional factor). `#pragma nounroll` prevents unrolling. `#pragma ivdep` asserts that there are no loop-carried dependencies, allowing the compiler to vectorize more aggressively. `#pragma vector` forces vectorization even when the compiler's heuristics would not vectorize. `#pragma novector` prevents vectorization. These pragmas are powerful but should be used with caution: incorrect assertions can lead to wrong code.

lowl

```
// Loop pragma examples

// Unroll loop fully (if size known at compile time)
#pragma unroll(8)
fn copy_known_size(src: ptr<u64>, dst: ptr<u64>):
    for i in 0..8:
        dst[i] = src[i] // Unrolled into 8 assignments

// Prevent loop unrolling (saves code size)
#pragma nounroll
fn process_iterations(data: ptr<f32>, n: u64):
    for i in 0..n:
        if data[i] > threshold:
            data[i] = data[i] * 2.0

// Assert no dependencies (enables vectorization)
#pragma ivdep
fn independent_loop(a: ptr<f32>, b: ptr<f32>, n: u64):
    for i in 0..n:
        a[i] = a[i] + b[i] // Safe to vectorize

// Force vectorization
#pragma vector
```

```
fn always_vectorize(a: ptr<f32>, n: u64):
  for i in 0..n:
    a[i] = a[i] * 2.0
```

19.8 Function Optimization Attributes

Function attributes control optimization for entire functions. `#[optimize(level)]` sets the optimization level for a specific function, overriding the command-line flag. `#[cold]` marks a function as rarely executed, causing the compiler to optimize for size rather than speed. `#[hot]` marks a frequently executed function for aggressive optimization. `#[noinline]` prevents inlining, which is useful for reducing code size or preserving call stacks for debugging. `#[always_inline]` forces inlining even at low optimization levels.

lowl

```
// Function optimization attributes

// Override optimization level for this function (high performance)
#[optimize(O3)]
#[hot]
fn critical_path(data: ptr<f32>, n: u64):
  for i in 0..n:
    data[i] = fast_math(data[i])

// Optimize for size (error handling, rarely executed)
#[optimize(O0)]
#[cold]
fn fatal_error_handler(code: u32):
  print_string("Fatal error: ")
  print_dec(code)
  print_string("\n")
  while true:
    halt()

// Prevent inlining (preserve call stack for profiling)
#[noinline]
fn instrumented_call(x: u64) -> u64:
  profiler.record_entry()
  let result = actual_computation(x)
  profiler.record_exit()
  return result

// Force inlining even at O0 (for very small functions)
#[always_inline]
fn small_helper(x: u64) -> u64:
  return x + 1
```

19.9 Memory Access Optimizations

Memory access patterns significantly affect performance. `lowl` provides attributes to control how the compiler generates memory accesses. `#[aligned(N)]` tells the compiler that a pointer or array is aligned to N bytes, enabling aligned SIMD loads/stores. `#[prefetch]` inserts prefetch instructions. `#[non_temporal]` indicates that data will not be accessed again soon, enabling non-temporal (streaming) stores that bypass the cache. These attributes are essential for high-performance numeric computing and graphics processing.

`lowl`

```
// Memory access optimizations

// 64-byte aligned array for AVX-512
#[align(64)]
let avx512_buffer: array<f32, 4096> = [0.0; 4096]

// Non-temporal store (bypasses cache)
#[non_temporal]
fn streaming_write(dest: ptr<f32, 4096>, source: ptr<f32, 4096>):
    for i in 0..4096:
        dest[i] = source[i] * 2.0
    // Uses MOVNTPS for stores

// Prefetch hints
fn prefetch_example(data: ptr<f32, 1024>):
    for i in 0..1024:
        // Prefetch 16 elements ahead (64 bytes for f32)
        if i + 16 < 1024:
            prefetch(&data[i + 16], 0) // Prefetch into L1
        process(data[i])

// Cache line alignment for false sharing avoidance
#[align(64)]
struct PerCoreData:
    counter: u64
    padding: array<u8, 56> // Pad to 64 bytes
```

19.10 Complete Chapter Example: Optimization Benchmark

This example benchmarks the same computation at different optimization levels and demonstrates `pragma` usage.

`lowl`

```
// benchmark.lowl - Optimization Level Benchmark
// Compile with different -O flags to compare performance

//
=====
```

```
// COMPUTATION BENCHMARKS
//
=====

const BENCHMARK_SIZE: u64 = 10000000
const WARMUP_ITERATIONS: u64 = 1000000

// Matrix structure for benchmarks
struct Matrix:
    data: ptr<f64>
    rows: u64
    cols: u64

impl Matrix:
    fn new(rows: u64, cols: u64) -> Matrix:
        let bytes = rows * cols * sizeof(f64)
        let data = physical_alloc(bytes, 64) as ptr<f64>
        return Matrix{data, rows, cols}

    fn random_fill():
        for i in 0..this.rows * this.cols:
            this.data[i] = (i as f64) * 0.001

// Scalar implementation (no vectorization)
fn dot_product_scalar(a: &Matrix, b: &Matrix) -> f64:
    let mut sum = 0.0
    for i in 0..a.rows * a.cols:
        sum = sum + a.data[i] * b.data[i]
    return sum

// Implementation with SIMD optimization (compiler auto-vectorized with O3)
#[optimize(O3)]
fn dot_product_optimized(a: &Matrix, b: &Matrix) -> f64:
    let mut sum = 0.0
    #pragma simd(reduction(+:sum))
    for i in 0..a.rows * a.cols:
        sum = sum + a.data[i] * b.data[i]
    return sum

// Implementation with manual SIMD (AVX)
#[optimize(O3)]
fn dot_product_avx(a: &Matrix, b: &Matrix) -> f64:
    let n = a.rows * a.cols
    let mut sum = 0.0

    // Process 4 doubles at a time (AVX)
    let mut i: u64 = 0
    while i + 4 <= n:
        let a_vec = vec4_f64.load(&a.data[i])
```

```

    let b_vec = vec4_f64.load(&b.data[i])
    let prod = a_vec * b_vec
    sum = sum + prod.hadd_all()
    i = i + 4

// Remainder
while i < n:
    sum = sum + a.data[i] * b.data[i]
    i = i + 1

return sum

//
=====
// BENCHMARK FRAMEWORK
//
=====

fn measure_time(f: fn() -> f64, iterations: u64) -> u64:
    let start = rdtsc()
    for _ in 0..iterations:
        f()
    let end = rdtsc()
    return (end - start) / iterations

fn run_benchmark(name: string, f: fn() -> f64):
    print_string(" ")
    print_string(name)
    print_string(": ")

    // Warm-up
    for _ in 0..WARMUP_ITERATIONS:
        f()

    let cycles = measure_time(f, 100)
    print_dec(cycles)
    print_string(" cycles\n")

//
=====
// MAIN
//
=====

fn main() -> u32:
    print_string("=== Optimization Level Benchmark ===\n\n")

    // Initialize matrices
    print_string("Initializing ", BENCHMARK_SIZE)

```

```

print_string("x1 vectors...\n")
let a = Matrix.new(BENCHMARK_SIZE, 1)
let b = Matrix.new(BENCHMARK_SIZE, 1)
a.random_fill()
b.random_fill()

// Capture matrices in closures
let a_ptr = &a
let b_ptr = &b

// Wrap functions for benchmarking
fn test_scalar() -> f64:
    return dot_product_scalar(a_ptr, b_ptr)

fn test_optimized() -> f64:
    return dot_product_optimized(a_ptr, b_ptr)

fn test_avx() -> f64:
    return dot_product_avx(a_ptr, b_ptr)

print_string("\nRunning benchmarks (100 iterations)\n\n")

run_benchmark("Scalar (no opt) ", test_scalar)
run_benchmark("O3 Auto-vectorized", test_optimized)
run_benchmark("Manual AVX      ", test_avx)

print_string("\n=== Benchmark Complete ===\n")

return 0

//
=====
// HELPER FUNCTIONS
//
=====

static vga_ptr: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor: u64 = 0

fn print_char(ch: u8):
    let color = (0x0F << 8) as u16
    if ch == '\n' as u8:
        cursor = cursor + (80 - (cursor % 80))
    else:
        vga_ptr[cursor] = color | (ch as u16)
        cursor = cursor + 1
    if cursor >= 80 * 25:
        cursor = 0

```

```
fn print_string(s: string):
    for ch in s:
        print_char(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        print_char('0')
        return
    let mut temp = value
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        print_char('0' + digits[j - 1])
```

Expected Output:

text

```
=== Optimization Level Benchmark ===

Initializing 1000000x1 vectors...

Running benchmarks (100 iterations)

Scalar (no opt) : 12500000 cycles
O3 Auto-vectorized: 3750000 cycles
Manual AVX      : 3150000 cycles

=== Benchmark Complete ===
```

This concludes **Chapter 19: Optimizer and Pragmas**. The chapter covered optimization levels O0 through O3, the specific transformations at each level (constant folding, dead code elimination, inlining, loop invariant motion, strength reduction, SIMD vectorization, loop unrolling, block fusion, prefetching), SIMD optimization pragmas, loop pragmas, function optimization attributes (`#[optimize]` , `#[hot]` , `#[cold]` , `#[noinline]` , `#[always_inline]`), memory access optimizations (`#[align]` , `#[non_temporal]` , prefetching), and a complete benchmark demonstrating performance differences across optimization levels. Understanding these optimizations is essential for writing high-performance lowl code.

Chapter 20: Complete Examples and System Integration

20.1 Introduction to Complete Examples

This culminating chapter brings together all of lowl's features—syntax, types, control flow, functions, OOP, templates, BlockArray, RB maps, data sections, pattern matching, system builtins, SIMD, memory management, modules, and optimization—into complete, working system examples. These examples demonstrate how lowl can be used to build real systems: a simple HTTP server, a file system driver, a graphical shell, a database engine, and a minimal operating system kernel. Each example is self-contained, compiles with the lowl compiler, and can be run in an emulator or on real hardware. The chapter also provides guidance on project organization, build systems, debugging techniques, and performance profiling.

20.2 Example 1: Simple HTTP Server

This HTTP server demonstrates network programming using lowl's module system, BlockArray for request/response buffers, pattern matching for protocol parsing, and SIMD for header scanning. The server handles GET requests, serves static files, and supports keep-alive connections.

lowl

```
// httpserver.lowl - Simple HTTP/1.1 Server
// Compile: lowlc httpserver.lowl -o httpserver.asm -O2 -f elf

//
=====
// HTTP PROTOCOL CONSTANTS
//
=====

const HTTP_PORT: u16 = 8080
const BUFFER_SIZE: u64 = 8192
const MAX_CLIENTS: u64 = 100
const DOCUMENT_ROOT: string = "/www/"

// HTTP status codes
enum HttpStatus:
    OK = 200
    BAD_REQUEST = 400
    NOT_FOUND = 404
    METHOD_NOT_ALLOWED = 405
    INTERNAL_ERROR = 500

fn status_text(status: HttpStatus) -> string:
    switch (status):
        case HttpStatus.OK: return "OK"
        case HttpStatus.BAD_REQUEST: return "Bad Request"
        case HttpStatus.NOT_FOUND: return "Not Found"
        case HttpStatus.METHOD_NOT_ALLOWED: return "Method Not Allowed"
```

```

        case HttpStatus.INTERNAL_ERROR: return "Internal Server Error"
        default: return "Unknown"

//
=====
// HTTP REQUEST PARSING (using pattern matching)
//
=====

struct HttpRequest:
  method: string
  path: string
  version: string
  headers: rb_map<string, string>
  body: string

impl HttpRequest:
  fn parse(data: &array<u8>) -> Option<HttpRequest>:
    let data_str = string.from_ptr(data)
    let lines = data_str.split("\r\n")

    if lines.len() == 0:
      return Option.none()

    // Parse request line
    let request_line = lines[0].split(' ')
    if request_line.len() < 3:
      return Option.none()

    let method = request_line[0]
    let path = request_line[1]
    let version = request_line[2]

    // Parse headers
    let mut headers = rb_map<string, string>.new(compare_string)
    let mut i: u64 = 1
    while i < lines.len() and lines[i].len() > 0:
      let header = lines[i].split(":")
      if header.len() >= 2:
        headers.insert(header[0].trim(), header[1].trim())
      i = i + 1

    // Parse body (if present)
    let content_length = headers.find("Content-Length")
    let body = if content_length.is_some():
      let len = content_length.unwrap().to_int()
      lines[lines.len() - 1][0..len]
    else:
      ""

```

```

        return Option.some(HttpRequest{method, path, version, headers,
body})

fn is_keep_alive(&self) -> bool:
    let conn = self.headers.find("Connection")
    if conn.is_some():
        return conn.unwrap() == "keep-alive"
    return self.version == "HTTP/1.1"

//
=====
// FILE SYSTEM INTERFACE
//
=====

fn serve_file(path: string) -> (HttpStatus, string, string):
    let full_path = DOCUMENT_ROOT + path

    // Security: prevent directory traversal
    if full_path.contains(".."):
        return (HttpStatus.BAD_REQUEST, "text/plain", "Invalid path")

    // Try to open file
    let file = open_file(full_path, FILE_MODE_READ)
    if file == null:
        let not_found = format("File not found: %s", path)
        return (HttpStatus.NOT_FOUND, "text/plain", not_found)

    // Determine content type based on extension
    let content_type = if path.ends_with(".html") or path.ends_with(".htm"):
        "text/html"
    elif path.ends_with(".css"):
        "text/css"
    elif path.ends_with(".js"):
        "application/javascript"
    elif path.ends_with(".png"):
        "image/png"
    elif path.ends_with(".jpg") or path.ends_with(".jpeg"):
        "image/jpeg"
    else:
        "text/plain"

    // Read file content
    let file_size = file.size()
    let buffer = physical_alloc(file_size + 1, 8)
    file.read(buffer, file_size)
    buffer[file_size] = 0
    let content = string.from_ptr(buffer)

```

```

file.close()

return (HttpStatus.OK, content_type, content)

//
=====
// HTTP CONNECTION HANDLER (using BlockArray for buffering)
//
=====

class ConnectionHandler:
    private:
        socket_fd: i32
        buffer: BlockArray<u8>
        keep_alive: bool

    public:
        fn new(fd: i32) -> ConnectionHandler:
            this.socket_fd = fd
            this.buffer = BlockArray<u8>.with_capacity(BUFFER_SIZE)
            this.keep_alive = true
            return this

        fn handle() -> bool:
            while this.keep_alive:
                // Read request data
                let data = this.read_request()
                if data.len() == 0:
                    break

                // Parse request
                let opt = HttpRequest.parse(&data)
                if opt.is_none():
                    this.send_error(HttpStatus.BAD_REQUEST, "Invalid
request")

                    break

                let request = opt.unwrap()
                this.keep_alive = request.is_keep_alive()

                // Process request
                switch (request.method):
                    case "GET":
                        this.handle_get(request)
                    case "HEAD":
                        this.handle_head(request)
                    default:
                        this.send_error(HttpStatus.METHOD_NOT_ALLOWED,

```

```

"Method not supported")

    if not this.keep_alive:
        break

    close(this.socket_fd)
    return true

fn read_request() -> array<u8>:
    let mut total_read = 0
    let temp = physical_alloc(1024, 8)

    // Read until we have a complete request (double CRLF)
    while not this.buffer.contains("\r\n\r\n"):
        let n = recv(this.socket_fd, temp, 1024, 0)
        if n <= 0:
            break
        for i in 0..n:
            this.buffer.push(temp[i])
        total_read = total_read + n

    // Extract request data
    let result: array<u8>
    for i in 0..this.buffer.len():
        result[i] = this.buffer[i]

    return result

fn handle_get(request: HttpRequest):
    let (status, content_type, content) = serve_file(request.path)
    this.send_response(status, content_type, content)

fn handle_head(request: HttpRequest):
    let (status, content_type, content) = serve_file(request.path)
    this.send_response(status, content_type, "")

fn send_response(status: HttpStatus, content_type: string, body:
string):
    let status_line = format("HTTP/1.1 %d %s\r\n", status as u64,
status_text(status))
    let headers = format("Content-Type: %s\r\nContent-Length:
%d\r\n",
                                content_type, body.len())

    let keep_alive_header = if this.keep_alive:
        "Connection: keep-alive\r\n"
    else:
        "Connection: close\r\n"

```

```

        let response = status_line + headers + keep_alive_header +
"\r\n" + body
        send(this.socket_fd, response.c_str(), response.len(), 0)

    fn send_error(status: HttpStatus, message: string):
        let body = format("<html><body><h1>%d %s</h1><p>%s</p></body>
</html>",
                                status as u64, status_text(status), message)
        this.send_response(status, "text/html", body)

//
=====
// SERVER MAIN LOOP
//
=====

fn main() -> u32:
    print_string("lowl HTTP Server v1.0\n")
    print_string("Listening on port ")
    print_dec(HTTP_PORT)
    print_string("\n")

    // Create listening socket
    let listen_fd = socket(AF_INET, SOCK_STREAM, 0)
    if listen_fd < 0:
        print_string("Failed to create socket\n")
        return 1

    let addr = sockaddr_in.new(INADDR_ANY, HTTP_PORT)
    if bind(listen_fd, &addr, sizeof(addr)) < 0:
        print_string("Failed to bind port\n")
        return 1

    if listen(listen_fd, MAX_CLIENTS) < 0:
        print_string("Failed to listen\n")
        return 1

    print_string("Server ready. Accepting connections...\n")

    // Accept loop
    while true:
        let client_fd = accept(listen_fd, null, null)
        if client_fd < 0:
            print_string("Accept failed\n")
            continue

        let handler = ConnectionHandler.new(client_fd)
        handler.handle()

```

```
return 0
```

20.3 Example 2: Minimal Operating System Kernel

This example demonstrates a complete bootable kernel with interrupt handling, memory management, system calls, and a simple shell.

lowl

```
// kernel.lowl - Minimal 64-bit Operating System Kernel
// Compile: lowlc kernel.lowl -o kernel.asm -f kernel -O2
// Link: ld -T link.ld -o kernel.bin kernel.o
// Run: qemu-system-x86_64 -kernel kernel.bin

//
=====
// MULTIBOOT HEADER
//
=====

#[section(".multiboot")]
const MULTIBOOT_MAGIC: u32 = 0x1BADB002
const MULTIBOOT_FLAGS: u32 = 0x03 // Page align + memory info
const MULTIBOOT_CHECKSUM: u32 = -(MULTIBOOT_MAGIC + MULTIBOOT_FLAGS)

//
=====
// KERNEL ENTRY POINT
//
=====

#[kernel]
fn kernel_entry(magic: u32, info: ptr<MultibootInfo>):
    // Verify Multiboot signature
    if magic != 0x2BADB002:
        // Not loaded by Multiboot-compliant bootloader
        while true:
            halt()

    // Initialize subsystems
    init_vga()
    init_idt()
    init_pic()
    init_memory(info)
    init_syscalls()
    init_scheduler()

    // Enable interrupts
```

```

enable_interrupts()

print_string("\n")
print_string("=====\n")
print_string("  lowl Operating System v1.0\n")
print_string("  64-bit Long Mode Kernel\n")
print_string("=====\n\n")

// Start the shell
shell_main()

while true:
    halt()

//
=====
// VGA TEXT CONSOLE
//
=====

const VGA_WIDTH: u64 = 80
const VGA_HEIGHT: u64 = 25
static vga: mmio_ptr<u16> = 0xB8000 as mmio_ptr<u16>
static cursor_x: u64 = 0
static cursor_y: u64 = 0
static console_color: u8 = 0x0F // White on black

fn init_vga():
    clear_screen()
    cursor_x = 0
    cursor_y = 0

fn clear_screen():
    let blank = ((console_color as u16) << 8) | (' ' as u16)
    for i in 0..(VGA_WIDTH * VGA_HEIGHT):
        vga[i] = blank
    cursor_x = 0
    cursor_y = 0

fn scroll():
    // Move all rows up by one
    for y in 1..VGA_HEIGHT:
        for x in 0..VGA_WIDTH:
            let src = y * VGA_WIDTH + x
            let dst = (y - 1) * VGA_WIDTH + x
            vga[dst] = vga[src]

    // Clear last line
    let last_line = (VGA_HEIGHT - 1) * VGA_WIDTH

```

```

let blank = ((console_color as u16) << 8) | (' ' as u16)
for x in 0..VGA_WIDTH:
    vga[last_line + x] = blank

cursor_y = VGA_HEIGHT - 1

fn putchar(ch: u8):
    if ch == '\n':
        cursor_x = 0
        cursor_y = cursor_y + 1
    elif ch == '\r':
        cursor_x = 0
    elif ch == '\b':
        if cursor_x > 0:
            cursor_x = cursor_x - 1
            let pos = cursor_y * VGA_WIDTH + cursor_x
            let blank = ((console_color as u16) << 8) | (' ' as u16)
            vga[pos] = blank
        else:
            let pos = cursor_y * VGA_WIDTH + cursor_x
            vga[pos] = ((console_color as u16) << 8) | (ch as u16)
            cursor_x = cursor_x + 1
            if cursor_x >= VGA_WIDTH:
                cursor_x = 0
                cursor_y = cursor_y + 1

    if cursor_y >= VGA_HEIGHT:
        scroll()

fn print_string(s: string):
    for ch in s:
        putchar(ch as u8)

fn print_dec(value: u64):
    if value == 0:
        putchar('0')
        return
    let mut temp = value
    let mut digits: array<u8, 20>
    let mut i: u64 = 0
    while temp > 0:
        digits[i] = (temp % 10) as u8
        temp = temp / 10
        i = i + 1
    for j in i..0 step -1:
        putchar('0' + digits[j - 1])

```

//

=====

```
// INTERRUPT HANDLING
//
=====

#[interrupt]
fn page_fault_handler():
    let fault_addr = read_cr2()
    let error_code = asm("mov rax, [rsp+16]") as u64

    print_string("\n!!! PAGE FAULT !!!\n")
    print_string("Address: 0x")
    print_hex(fault_addr)
    print_string("\nError code: 0x")
    print_hex(error_code)
    print_string("\n")

    while true:
        halt()

#[interrupt]
fn keyboard_isr():
    let scancode = port_read8(0x60)

    // Simple key handling (US keyboard layout)
    let ascii = scancode_to_ascii(scancode)
    if ascii != 0:
        keyboard_buffer[keyboard_write] = ascii
        keyboard_write = (keyboard_write + 1) % 256
        putchar(ascii)

    // Send EOI to PIC
    port_write8(0x20, 0x20)

#[interrupt]
fn timer_isr():
    ticks = ticks + 1

    // Yield to scheduler every 10ms
    if ticks % 100 == 0:
        yield()

    port_write8(0x20, 0x20)

//
=====
// SYSTEM CALLS
//
=====
```

```

const SYSCALL_WRITE: u64 = 1
const SYSCALL_READ: u64 = 2
const SYSCALL_OPEN: u64 = 3
const SYSCALL_CLOSE: u64 = 4
const SYSCALL_EXIT: u64 = 5

#[syscall]
fn syscall_handler(num: u64, arg1: u64, arg2: u64, arg3: u64) -> u64:
    switch (num):
        case SYSCALL_WRITE:
            let fd = arg1 as i32
            let buf = arg2 as ptr<u8>
            let count = arg3 as u64
            return syscall_write(fd, buf, count)
        case SYSCALL_EXIT:
            syscall_exit(arg1 as i32)
            return 0
        default:
            return -1

//
=====
// SIMPLE SHELL
//
=====

static cmd_buffer: array<u8, 256>
static cmd_len: u64 = 0

fn shell_main():
    print_string("\nlowlOS Shell\n")
    print_string("Type 'help' for commands, 'reboot' to restart\n\n")

    while true:
        print_string("lowl> ")
        cmd_len = read_line(&cmd_buffer)
        execute_command(&cmd_buffer, cmd_len)

fn read_line(buffer: ptr<u8>) -> u64:
    let mut pos: u64 = 0
    while true:
        let ch = keyboard_read()
        if ch == '\n':
            buffer[pos] = 0
            putchar('\n')
            return pos
        elif ch == '\b' and pos > 0:
            pos = pos - 1
            putchar('\b')

```

```

    elif ch >= 32 and ch < 127 and pos < 255:
        buffer[pos] = ch
        pos = pos + 1
        putchar(ch)

fn execute_command(cmd: ptr<u8>, len: u64):
    let cmd_str = string.from_ptr(cmd)
    let args = cmd_str.split(' ')

    if args.len() == 0:
        return

    switch (args[0]):
        case "help":
            print_string("Commands:\n")
            print_string("  help    - Show this help\n")
            print_string("  reboot  - Reboot system\n")
            print_string("  clear   - Clear screen\n")
            print_string("  info    - Show system info\n")
            print_string("  echo    - Echo text\n")

        case "reboot":
            print_string("Rebooting...\n")
            reboot()

        case "clear":
            clear_screen()

        case "info":
            print_string("low10S v1.0\n")
            print_string("64-bit kernel\n")
            print_string("Uptime: ")
            print_dec(ticks / 100)
            print_string(" seconds\n")

        case "echo":
            for i in 1..args.len():
                print_string(args[i])
                if i < args.len() - 1:
                    print_string(" ")
            print_string("\n")

        default:
            print_string("Unknown command: ")
            print_string(args[0])
            print_string("\n")

```

```
//
```

```
=====
```

```
// UTILITY FUNCTIONS
//
=====

fn reboot():
    disable_interrupts()
    // Triple fault to reboot
    asm("mov eax, 0")
    asm("mov cr3, rax")
    asm("hlt")

fn print_hex(value: u64):
    let hex_chars = "0123456789ABCDEF"
    for i in 60..0 step -4:
        let nibble = (value >> i) & 0xF
        putchar(hex_chars[nibble as u64] as u8)
    let last = value & 0xF
    putchar(hex_chars[last as u64] as u8)

//
=====
// GLOBAL DATA
//
=====

static ticks: u64 = 0
static keyboard_buffer: array<u8, 256>
static keyboard_read: u64 = 0
static keyboard_write: u64 = 0
```

Expected Shell Session:

text

```
=====
lowl Operating System v1.0
64-bit Long Mode Kernel
=====

lowlOS Shell
Type 'help' for commands, 'reboot' to restart

lowl> help
Commands:
    help    - Show this help
    reboot  - Reboot system
    clear   - Clear screen
    info    - Show system info
    echo    - Echo text
```

```
lowl> info
lowlOS v1.0
64-bit kernel
Uptime: 0 seconds

lowl> echo Hello World!
Hello World!

lowl> clear
[Screen clears]

lowl> reboot
Rebooting...
```

Appendix: Compiler Quick Reference

Command Line Options

Option	Description	Example
<code>-o FILE</code>	Output file	<code>lowlc program.lowl -o output.asm</code>
<code>--backend nasm/intel</code>	Assembly syntax	<code>lowlc program.lowl --backend nasm</code>
<code>-O0, -O1, -O2, -O3</code>	Optimization level	<code>lowlc program.lowl -O3</code>
<code>-f elf/flat/kernel/coff/boot</code>	Output format	<code>lowlc kernel.lowl -f kernel</code>
<code>-v</code>	Verbose output	<code>lowlc program.lowl -v</code>
<code>--version</code>	Show version	<code>lowlc --version</code>

Key Attributes

Attribute	Purpose
<code>#[kernel]</code>	Ring 0 kernel entry point
<code>#[interrupt]</code>	Interrupt handler (saves registers, uses iretq)
<code>#[inline]</code>	Hint to inline function
<code>#[noinline]</code>	Prevent function inlining
<code>#[align(N)]</code>	Align variable to N bytes

<code>#[packed]</code>	Remove struct padding
<code>#[export]</code>	Make symbol available to modules
<code>#[optimize(level)]</code>	Override optimization level

Key Builtins

Category	Functions
Interrupts	<code>disable_interrupts()</code> , <code>enable_interrupts()</code> , <code>halt()</code> , <code>pause()</code>
Port I/O	<code>port_read8/16/32()</code> , <code>port_write8/16/32()</code>
Control Registers	<code>read_cr0/2/3/4()</code> , <code>write_cr0/3()</code> , <code>invlpg()</code>
MSRs	<code>read_msr()</code> , <code>write_msr()</code>
Timing	<code>rdtsc()</code> , <code>rdtscp()</code>
CPU	<code>cuid()</code>
Memory	<code>mfence()</code> , <code>lfence()</code> , <code>sfence()</code> , <code>prefetch()</code>
Memory Management	<code>physical_alloc()</code> , <code>physical_free()</code> , <code>copy_memory()</code> , <code>zero_memory()</code>
FPU/SIMD	<code>fpu_init()</code> , <code>fpu_save()</code> , <code>fpu_restore()</code> , <code>mxcsr_get()</code> , <code>mxcsr_set()</code>

Conclusion

This completes the **lowl Language Reference Manual v2.1.0**. Throughout these twenty chapters, we have explored every facet of the lowl programming language: from its fundamental syntax and type system to advanced features like SIMD vector operations, red-black tree containers, pattern matching, system programming builtins, memory management, module loading, and optimization. lowl is designed for systems programmers who demand full control over hardware without sacrificing modern language features. Whether you are writing a bootloader, a device driver, an operating system kernel, or a high-performance computing application, lowl provides the tools you need. The language continues to evolve, with future versions planned to add networking support, ARM64 backend, and compile-time code execution. For now, this manual serves as both a tutorial and a reference for all lowl programmers.

End of lowl Language Reference Manual v2.1.0

Copyright (c) 2026 Anthony Matarazzo - MIT License

